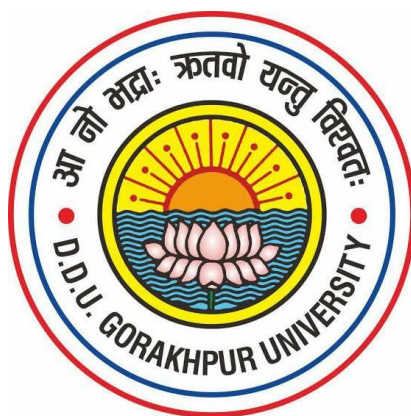




INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

INSTITUTE OF ENGINEERING AND TECHNOLOGY
DEEN DAYAL UPADHYAYA GORAKHPUR UNIVERSITY,
GORAKHPUR
(दीन दयाल उपाध्याय गोरखपुर विश्वविद्यालय, गोरखपुर)



COURSE STRUCTURE
&
SYLLABUS

FOR

B. TECH.

Computer Science & Engineering

Based

on

AICTE MODEL CURRICULUM

[Effective from the Session: 2024-25]



INSTITUTE OF ENGINEERING AND TECHNOLOGY

Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

University offers a pool of courses for Ability Enhancement Courses (AEC) and Skill Enhancement Courses (SEC). Student has to select one SEC course (repetition of courses are not allowed) in first, second and third semester respectively. Further student has to select one AEC course (repetition of courses are not allowed) in first, second, third and fourth semester respectively.



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Curriculum for Bachelor of Technology
COMPUTER SCIENCE AND ENGINEERING
Course Structure & Semester-wise Credit Distribution

Credit Distribution (Course Type) in B. Tech. (Computer Science & Engineering) Program		
S.No.	Course Type	Credit
1	BSC	14
2	ESC	19
3	HMSC	10
4	PCC	69
5	PEC	18
6	Project work, Seminar and internship in industry or elsewhere	16
7	AEC offered by University	08
8	SEC offered by University	09
9	OEC	12
10	Mandatory Non-Credit Courses [ACITE-Mandatory Non-Credit (NC) Courses]	(02) Non-Credit
Total Credit		175

Abbreviation:

BSC: Basic Science Course

ESC: Engineering Science Course

HMSC: Humanities & Social Science including Management Courses

PCC: Professional Core Course

PEC: Professional Elective Course

OEC: Open Elective Course



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Basic Science Courses (BSC)			
S. No.	Course Code	Course Title	Credits
1.	ECHE101	Engineering Chemistry	3+0
2.	ECHE151	Engineering Chemistry Lab	0+1
3.	EPHY101	Engineering Physics	3+0
4.	EPHY151	Engineering Physics Lab	0+1
5.	EMAT101	Engineering Mathematics-I	3+0
6.	EMAT 102	Engineering Mathematics-II	3+0
Total			14
Engineering Science Course (ESC)			
1.	ECE101	Basic Electronics Engineering	3+0
2.	ECE151	Basic Electronics Engineering Lab	0+1
3.	ECE102	Basic Electrical Engineering	3+0
4.	ECE152	Basic Electrical Engineering Lab	0+1
5.	ME101	Engineering Graphics & Design	0+2
6.	CSE101	Programming for Problem Solving	3+0
7.	CSE101	Programming for Problem Solving Lab	0+1
8.	ME102	Workshop Practices	0+2
9.	ME103	Fundamental of Mechanical Engineering and Mechatronics	3+0
Total			19
Humanities and Social Sciences including Management Courses (HSMC)			
1.	HSM101	Professional Communication	3+0
2.	HSM301	Organization Behavior	3+0
3.	HUV102	Universal Human Values-II: Understanding Harmony And Ethical Human Conduct	3+0
4.	HSM151	Professional Communication Lab	0+1
Total			10



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Professional Core Course (PCC)			
S.No.	Course Code	Course Title	Credit
1.	CSE201	Mathematics for Machine Learning	3+0
2.	CSE251	Mathematics for Machine Learning Lab	0+1
3.	ECE201	Digital Electronics & Logic Design	3+0
4.	ECE251	Digital Electronics & Logic Design Lab	0+1
5.	CSE202	Data Structure	4+0
6.	CSE252	Data Structure Lab	0+1
7.	CSE203	Python with Linux	2+0
8.	CSE253	Python with Linux Lab	0+1
9.	CSE204	Discrete Mathematics	4+0
10.	CSE205	Evolutionary Computing	3+0
11.	CSE255	Evolutionary Computing Lab	0+1
12.	CSE206	Computer Organization and Architecture	3+0
13.	CSE256	Computer Organization and Architecture Lab	0+1
14.	CSE207	Operating Systems	4+0
15.	CSE257	Operating System Lab	0+1
16.	CSE208	Software Engineering	3+0
17.	CSE258	Software Engineering Lab	0+1
18.	CSE209	Theory of Automata and Formal Languages	4+0
19.	CSE301	Artificial Intelligence	4+0
20.	CSE302	Database Management System	3+0
21.	CSE352	Database Management System Lab	0+1
22.	CSE303	Design and Analysis of Algorithms	4+0
23.	CSE353	Design and Analysis of Algorithm Lab	0+1
24.	CSE304	Computer Network	4+0
25.	CSE354	Computer Network Lab	0+1
26.	CSE305	Compiler Design	3+0
27.	CSE355	Compiler Design Lab	0+1
28.	CSE306	Machine Learning	4+0
29.	CSE356	Machine Learning Lab	0+1
30.	IT355	Internet Programming using Java	0+1
Total			69

Computer Science & Engineering Department Project Work, Seminar and Internship/Industrial Training in Industry or Elsewhere			
S.No.	Course Code	Course Title	Credit
1.	CSEP312	Project I	0+3
2.	CSEIND401	Industrial Training	0+2
3.	CSES401	Seminar	0+2
4.	CSEP402	Project II	0+3
5.	CSEP403	Project III	0+6
Total			16



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Professional Elective Course (PEC)			
S.No.	Semester	PEC-Elective	Credit
1	V	Elective-I	3+0
2	VI	Elective-II	3+0
3	VI	Elective-II	3+0
4	VII	Elective-I	3+0
5	VII	Elective-I	3+0
6	VIII	Elective-II	3+0
Total			18
Student has to select unique course(s) either from Elective-I or from Elective-II in the semester. Repetition of courses is not allowed.			



INSTITUTE OF ENGINEERING AND TECHNOLOGY

Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Professional Elective Courses (from SWAYAM)

Discipline: Computer Science and Engineering, **Duration:** 12 Weeks, **Credit:** 3

Elective-1 (Odd Semester)

S.No.	Course Code	Course Id	Course Name	SME Name	Institute	Link
1.	ECSE101	noc23-cs127	Cyber Security and Privacy	Prof. Saji K Mathew	IITM	https://onlinecourses.nptel.ac.in/noc23_cs127/preview
2.	ECSE102	noc23-cs77	Computer Vision	Prof. Jayanta Mukhopadhyay	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs77/preview
3.	ECSE103	noc23-cs89	Cloud Computing	Prof. Soumya Kanti Ghosh	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs89/preview
4.	ECSE104	noc23-cs114	C-Based VLSI Design	Prof. Chandan Karfa	IITG	https://onlinecourses.nptel.ac.in/noc23_cs114/preview
5.	ECSE105	noc23-cs92	Artificial Intelligence : Search Methods For Problem Solving	Prof. Deepak Khemani	IITM	https://onlinecourses.nptel.ac.in/noc23_cs92/preview
6.	ECSE106	noc23-cs102	Parameterized Algorithms	Prof. Neeldhara Misra	noc23-cs102	https://onlinecourses.nptel.ac.in/noc23_cs102/preview
7.	ECSE107	noc23-cs107	Social Networks	Prof. Sudarshan Iyengar Prof. Yayati Gupta	IIT Ropar	https://onlinecourses.nptel.ac.in/noc23_cs107/preview
8.	ECSE108	noc23-cs111	Computational Complexity	Prof. Subrahmanyam Kalyanasundaram	IIT Hyderabad	https://onlinecourses.nptel.ac.in/noc23_cs111/preview
9.	ECSE109	noc23-cs68	Advanced Distributed Systems	Prof. Smruti Ranjan Sarangi	IITD	https://onlinecourses.nptel.ac.in/noc23_cs68/preview
10.	ECSE110	noc23-cs84	Statistical Learning For Reliability Analysis	Prof. Monalisa Sarma	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs84/preview
11.	ECSE111	noc23-cs75	Ethical Hacking	Prof. Indranil Sengupta	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs75/preview
12.	ECSE112	noc23-cs91	Software Testing	Prof. Meenakshi D'Souza	IIITB	https://onlinecourses.nptel.ac.in/noc23_cs91/preview
13.	ECSE113	noc23-cs126	Deep Learning for Computer Vision	Prof. Vineeth N Balasubramanian	IITH	https://onlinecourses.nptel.ac.in/noc23_cs126/preview
14.	ECSE114	noc23-cs69	Privacy And Security In Online Social Media	Prof. Ponnurangam Kumaraguru	IITH	https://onlinecourses.nptel.ac.in/noc23_cs69/preview

Note: In case of changes in the above subjects (**Elective-1 (Even Semester) (from SWAYAM)**), Dean /Director/ HoD will issue separate list of subjects.



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Professional Elective Courses (from Department of CSE)

Elective-1 (Odd Semester)

S.No.	Course Code	Course Name	Credit
1.	ECSE115	Data Mining and Data Warehousing	3+0
2.	ECSE116	Advanced Database Management System	3+0
3.	ECSE117	Design and Analysis of Parallel Algorithms	3+0
4.	ECSE118	Computational Intelligence	3+0
5.	ECSE119	Parallel and Distributed Computing	3+0
6.	ECSE120	Natural Language Processing	3+0
7.	ECSE121	High Performance Computing	3+0
8.	ECSE122	Design & Development of Applications	3+0
9.	ECSE123	Introduction to Software Testing	3+0
10.	ECSE124	Distributed System	3+0
11.	ECSE125	Mobile Computing	3+0
12.	ECSE126	Wireless Network Systems	3+0

Note:

- Instead of adopting courses from Elective-I in Vth Semester, student may adopt “Internet Programming using Java (IT305)” offered by department of Information Technology.
- Students may take subjects from Elective-1 (from SWAYAM) and/or Elective-1 (from Department of CSE)
- Repetition of courses is not allowed.
- Execution of the courses offered by the department depends on the availability of faculty persons.



INSTITUTE OF ENGINEERING AND TECHNOLOGY

Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Elective-2 (Even Semester) (from SWAYAM)

Discipline: Computer Science and Engineering, **Duration:** 12 Weeks, **Credit:** 3

S.No.	Course Code	Course Id	Course Name	SME Name	Institute	Link
1.	ECSE201	noc23-cs07	Advanced Computer Architecture	Prof. Smruti R.Sarangi	IITD	https://onlinecourses.nptel.ac.in/noc23_cs07
2.	ECSE202	noc23-cs59	Hardware Security	Prof. Debdeep Mukhopadhyay	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs59
3.	ECSE203	noc23-cs61	GPU Architectures And Programming	Prof. Soumyajit Dey	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs61
4.	ECSE204	noc23-cs62	Foundations of Cyber Physical Systems	Prof. Soumyajit Dey	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs62
5.	ECSE205	noc23-cs63	Selected Topics in Algorithms	Prof. Palash Dey	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs63
6.	ECSE206	noc23-cs36	Affective Computing	Prof. Jainendra Shukla Prof. Abhinav Dhall	IITD	https://onlinecourses.nptel.ac.in/noc23_cs36
7.	ECSE207	noc23-cs51	Introduction To Internet Of Things	Prof. Sudip Misra	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs51
8.	ECSE208	noc23-cs08	Data Analytics with Python	Prof. A. Ramesh	IITR	https://onlinecourses.nptel.ac.in/noc23_cs08
9.	ECSE209	noc23-cs03	Foundations of Cryptography	Prof. Ashish Choudhury	IIT Bangalore	https://onlinecourses.nptel.ac.in/noc23_cs03
10.	ECSE210	noc23-cs23	Reinforcement Learning	Prof. Balaraman Ravindran	IITM	https://onlinecourses.nptel.ac.in/noc23_cs23
11.	ECSE211	noc23-cs24	Deep Learning	Prof. Sudarshan Iyengar	IIT Ropar	https://onlinecourses.nptel.ac.in/noc23_cs24
12.	ECSE212	noc23-cs45	Natural Language Processing	Prof. Pawan Goyal	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs45
13.	ECSE213	noc23-cs47	Blockchain and its Applications	Prof. Sandip Chakraborty Prof. Shamik Sural	IITKGP	https://onlinecourses.nptel.ac.in/noc23_cs47

Note: In case of changes in the above subjects (**Elective-2 (Even Semester) (from SWAYAM)**), Dean /Director/ HoD will issue separate list of subjects.



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Professional Elective Courses (from Department of CSE)

Elective-2 (Even Semester)

S.No.	Course Code	Course Name	Credit
1.	ECSE214	Randomized Algorithms	3+0
2.	ECSE215	Programming for Embedded Systems	3+0
3.	ECSE216	Introduction to Deep Learning	3+0
4.	ECSE217	Service Oriented Architecture	3+0
5.	ECSE218	Quantum Computing	3+0
6.	ECSE219	Introduction to Cloud Computing	3+0
7.	ECSE220	Blockchain Architecture Design	3+0
8.	ECSE221	Distributed Algorithms	3+0
9.	ECSE222	High Speed Networks	3+0
10.	ECSE223	Software Defined Network	3+0
11.	ECSE224	Pervasive Computing	3+0
12.	ECSE225	Middleware Technologies	3+0

Note:

- Instead of adopting courses from Elective-II in VIth Semester, student may adopt “**Neural Networks (ITE115)**” and “**Advanced Topics in Data Mining (IT308)**” offered by department of Information Technology.
- Students may take subjects from Elective-2 (from SWAYAM) and/or Elective-2 (from Department of CSE)
- Repetition of courses is not allowed.
- Execution of the courses offered by the department depends on the availability of faculty persons.



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

ACITE-Mandatory Non-Credit (NC) Courses

S. No.	Course Code	Course Title	Credits
1.	-----	Induction Program (3-weeks)	-----
2.	ENV201	Environment & Ecology	2+0

AICTE Mandatory 3-Week Student Induction Program

The Essence and Details of Induction program can also be understood from the 'Detailed Guide on Student Induction program', as available on AICTE Portal.

(Link: https://www.aicte-india.org/sites/default/files/Model_Curriculum/UG-1/ug-vol1.pdf).

Induction Program (mandatory)	Three-week duration
Induction Program for students to be offered right at the start of the first year.	Physical activity
	Creative Arts
	Universal Human Values-I
	Literary
	Proficiency Modules
	Lectures by Eminent People
	Visits to local Areas
	Familiarization to Dept./Branch & Innovations



INSTITUTE OF ENGINEERING AND TECHNOLOGY

Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

AEC and SEC Offered by the University for Implementation of NEP2020 (University Mandatory Course)

University offers a pool of courses for AEC and SEC to implement NEP2020. Student has to select one course under SEC category in first, second and third semester (repetition of courses are not allowed). In the same context, student has to select one course under AEC category in first, second, third and fourth semester (repetition of courses are not allowed).

As per NEP2020, year wise credit requirements for the award of “Certificate in Computer Science & Engineering”, “Diploma in Computer Science & Engineering”, “Bachelor of Vocation (B. Voc.) in Computer Science & Engineering” and “B. Tech. in Computer Science & Engineering” are given below:

First Year: After earning 47 credits in first year (25 credits in first semester and 22 credits in second semester), student will be eligible for “**Certificate in Computer Science & Engineering**”.

Second Year: After earning 95 credits (47 credits from first year and 48 credits in second year (25 credits in third semester and 23 credits in fourth semester)), student will be eligible for the award of “**Diploma in Computer Science & Engineering**”.

Third Year: After earning 144 credits (47 credits from first year and 48 credits in second year and 49 credits in third year (25 credits in fifth semester and 24 credits in sixth semester)), student will be eligible for the award of “**Bachelor of Vocation (B. Voc.) in Computer Science & Engineering**”.

Fourth Year: After earning 175 credits (47 credits from first year and 48 credits in second year, 49 credits in third year and 31 credit in fourth year (16 credits in seventh semester and 15 credits in eighth semester)), student will be eligible for the degree of “**B. Tech. in Computer Science & Engineering**”.

Table1: Year wise credit requirement for award of Certificate, Diploma, B. Voc., and B. Tech. Degree in Department of Computer Science & Engineering under the light of NEP2020.

After Year	Credit Requirement	Eligibility of
1 st	47	“Certificate in Computer Science & Engineering”
2 nd	95	“Diploma in Computer Science & Engineering”
3 rd	144	“Bachelor of Vocation (B. Voc.) in Computer Science & Engineering”
4 th	175	“B. Tech. in Computer Science & Engineering”

Skill Enhancement Course offered by Department of Computer Science & Engineering

1. Department of Computer Science & Engineering offers a course on “**Introduction to 3D Printing Technology (SEC-CSE01)**” with credit 03 under **Skill Enhancement Course** category.



INSTITUTE OF ENGINEERING AND TECHNOLOGY

Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Engineering Open Elective Courses

S. No.	Department	Course Code	Course Title	Credits	Category
1.	ECE	ECOE01	Introduction to Microprocessors and Embedded Systems	3+0	Engineering Open Elective Course-1
	IT	ITOE01	Introduction to OOP in C++		
	CSE	CSEOE01	Web Technology		
	ME	MEOE01	Renewable Energy Resources		
2.	ECE	ECOE02	Introduction to MEMs	3+0	Engineering Open Elective Course-2
	IT	ITOE02	Introduction to Virtualization and Cloud Computing		
	CSE	CSEOE02	Web Application Development using Python		
	ME	MEOE02	Robotics		
3.	ECE	ECOE03	Digital VLSI Design	3+0	Engineering Open Elective Course-3
	IT	ITOE03	Cyber Law and Ethics		
	CSE	CSEOE03	Front End Technologies		
	ME	MEOE03	Finite Element Method		
4.	ECE	ECOE04	Wireless Communication and Network	3+0	Engineering Open Elective Course-4
	IT	ITOE04	Internet of Things		
	CSE	CSEOE04	Back-End Technologies		
	ME	MEOE04	Operation Research		

Note:

- Students of the department may choose Engineering Open Elective Courses offered by CSE department or other departments also.
- Execution of the courses offered by the department depends on the availability of faculty persons.



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Credit Distribution (Semester wise) (Minimum Common Programme)

S.No.	Semester Number	Credit
1	Semester I	25
2	Semester II	22
3	Semester III	25
4	Semester IV	23
5	Semester V	25
6	Semester VI	24
7	Semester VII	16
8	Semester VIII	15
Total Credit		175

Note: Students, who have joined the job/internship/ summer training, can earn credits from MOOCS courses in the semesters.



Course Structure



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Branch/Course: Computer Science & Engineering
Semester-I (First Year) Curriculum

S. No.	Type of Course	Course Code	Course Title	Credit
1.	Basic Science Course	EPHY101	Engineering Physics	3+0
2.	Basic Science Course	EPHY151	Engineering Physics Lab	0+1
3.	Basic Science Course	EMAT 101	Engineering Mathematics-I	3+0
4.	Basic Engineering Course	CSE101	Programming for Problem Solving	3+0
5.	Basic Engineering Course	CSE151	Programming for Problem Solving Lab	0+1
6.	Basic Engineering Course	ECE102	Basic Electrical Engineering	3+0
7.	Basic Engineering Course	ECE152	Basic Electrical Engineering Lab	0+1
8.	Basic Engineering Course	ME102	Workshop Practices	0+2
9.	Basic Engineering Course	ME103	Fundamental of Mechanical Engineering and Mechatronics	3+0
10.	SEC Course	SEC-x	SEC Course	03
11.	AEC Course	AEC-x	AEC Course	02
12.	Induction Program	-----		-----
Total Credit				25

Semester-II (First Year) Curriculum

S. No.	Type of Course	Course Code	Course Title	Credit
1.	Basic Science Course	ECHE101	Engineering Chemistry	3+0
2.	Basic Science Course	ECHE151	Engineering Chemistry Lab	0+1
3.	Basic Science Course	EMAT102	Engineering Mathematics-II	3+0
4.	Humanities and Social Sciences including Management Course	HSM101	Professional Communication	3+0
5.	Humanities and Social Sciences including Management Course	HSM151	Professional Communication Lab	0+1
6.	Basic Engineering Course	ECE101	Basic Electronics Engineering	3+0
7.	Basic Engineering Course	ECE151	Basic Electronics Engineering Lab	0+1
8.	Basic Engineering Course	ME101	Engineering Graphics & Design	0+2
9.	SEC Course	SEC-x	SEC Course	03
10.	AEC Course	AEC-x	AEC Course	02
Total Credit				22

Note: After earning 47 credits in first year (25 credits in first semester and 22 credits in second semester), student will be eligible for “Certificate in Computer Science & Engineering”.



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Branch/Course: Computer Science & Engineering
Semester-III (Second Year) Curriculum

S. No.	Type of Course	Course Code	Course Title	Credit
1.	Professional Core Course	CSE201	Mathematics for Machine Learning	3+0
2.	Professional Core Course	CSE202	Data Structure	4+0
3.	Professional Core Course	CSE203	Python with Linux	2+0
4.	Professional Core Course	CSE204	Discrete Mathematics	4+0
5.	Professional Core Course	ECE202	Digital Electronics & Logic Design	3+0
6.	Professional Core Course	CSE251	Mathematics for Machine Learning Lab	0+1
7.	Professional Core Course	CSE252	Data Structure Lab	0+1
8.	Professional Core Course	CSE253	Python with Linux Lab	0+1
9.	Professional Core Course	ECE252	Digital Electronics & Logic Design Lab	0+1
10.	Mandatory Non-Credit-Course	ENV201	Environment & Ecology	2+0(Non-Credit)
11.	SEC Course	SEC-x	SEC Course	03
12.	AEC Course	AEC-x	AEC Course	02
Total Credit				25

Semester-IV (Second Year) Curriculum

S. No.	Type of Course	Course Code	Course Title	Credit
1.	Professional Core Course	CSE205	Evolutionary Computing	3+0
2.	Professional Core Course	CSE206	Computer Organization & Architecture	3+0
3.	Professional Core Course	CSE207	Operating Systems	4+0
4.	Professional Core Course	CSE208	Software Engineering	3+0
5.	Professional Core Course	CSE209	Theory of Automata and Formal Languages	4+0
6.	Professional Core Course	CSE255	Evolutionary Computing Lab	0+1
7.	Professional Core Course	CSE256	Computer Organization & Architecture Lab	0+1
8.	Professional Core Course	CSE257	Operating System Lab	0+1
9.	Professional Core Course	CSE258	Software Engineering Lab	0+1
10.	AEC Course	AEC-x	AEC Course	02
Total Credit				23

Note: After earning 95 credits (47 credits from first year and 48 credits in second year (25 credits in third semester and 23 credits in fourth semester)), student will be eligible for the award of “**Diploma in Computer Science & Engineering**”.



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Branch/Course: Computer Science & Engineering
Semester-V (Third Year) Curriculum

S. No.	Type of Course	Course Code	Course Title	Credit
1.	Professional Core Course	CSE301	Artificial Intelligence	4+0
2.	Professional Core Course	CSE302	Database Management System	3+0
3.	Professional Core Course	CSE303	Design and Analysis of Algorithm	4+0
4.	Professional Core Course	CSE304	Computer Network	4+0
5.	Professional Elective Courses		Elective-I (from SWAYAM/ Department of CSE)	3+0
6.	Humanities & Social Science including Management Courses	HSM301	Organization Behavior	3+0
7.	Professional Core Course	CSE352	Database Management System Lab	0+1
8.	Professional Core Course	CSE353	Design and Analysis of Algorithm Lab	0+1
9.	Professional Core Course	CSE354	Computer Network Lab	0+1
10.	Professional Core Course	IT355	Internet Programming using Java	0+1
Total Credit				25

Semester-VI (Third Year) Curriculum

S. No.	Type of Course	Course Code	Course Title	Credit
1.	Professional Core Course	CSE305	Compiler Design	3+0
2.	Professional Core Course	CSE306	Machine Learning	4+0
3.	Professional Elective Course		Elective-II (from SWAYAM/ Department of CSE)	3+0
4.	Professional Elective Course		Elective-II (from SWAYAM/ Department of CSE)	3+0
5.	Open Elective Course		Open Elective-I	3+0
6.	Humanities & Social Science including Management Courses	HUV102	Universal Human Values-II: Understanding Harmony and Ethical Human Conduct	3+0
7.	Professional Core Course	CSE355	Compiler Design Lab	0+1
8.	Professional Core Course	CSE356	Machine Learning Lab	0+1
9.	Professional Core Course	CSEP312	Project-I	0+3
Total Credit				24

Note: After earning 144 credits (47 credits from first year and 48 credits in second year and 49 credits in third year (25 credits in fifth semester and 24 credits in sixth semester)), student will be eligible for the award of **"Bachelor of Vocation (B. Voc.) in Computer Science & Engineering"**.



INSTITUTE OF ENGINEERING AND TECHNOLOGY
Deen Dayal Upadhyaya Gorakhpur University, Gorakhpur

Branch/Course: Computer Science & Engineering
Semester-VII (Fourth Year) Curriculum

S. No.	Type of Course	Course Code	Course Title	Credit
1.	Professional Elective Course		Elective-I (from SWAYAM/ Department of CSE)	3+0
2.	Professional Elective Course		Elective-I (from SWAYAM/ Department of CSE)	3+0
3.	Open Elective Courses		Open Elective-II	3+0
4.	Professional Core Course	CSES401	Seminar	0+2
5.	Professional Core Course	CSEIND401	Industrial Training	0+2
6.	Professional Core Course	CSEP402	Project-II	0+3
Total Credit				16

Semester-VIII (Fourth Year) Curriculum

S. No.	Type of Course	Course Code	Course Title	Credit
1.	Professional Elective Courses		Elective-II (from SWAYAM/ Department of CSE)	3+0
2.	Open Elective Courses		Open Elective-III	3+0
3.	Open Elective Courses		Open Elective-IV	3+0
4.	Professional Core Subject	CSEP403	Project-III	0+6
Total Credit				15

Note: After earning 175 credits (47 credits from first year and 48 credits in second year ,49 credits in third year and 31 credit in fourth year (16 credits in seventh semester and 15 credits in eighth semester)), student will be eligible for the degree of **“B. Tech. in Computer Science & Engineering”**.

Programming for Problem Solving (CSE101)

Credit: 3+0

Course Objective: Students will be able to develop logics which will help them to create programs, applications in C also by learning the basic programming constructs they can easily switch over to any other language in future.

Syllabus

- 1. Introduction to Programming:** Introduction to components of a computer system, Algorithm: Representation of Algorithm, Flowchart, Pseudo code with examples, from algorithms to programs, source code. Programming Basics: Structure of C program: writing and executing the first C program, Syntax and logical errors in compilation, object and executable code. Components of C language: Standard I/O in C, Fundamental data types, Variables and memory locations, Storage classes. Arithmetic expressions & Conditional Branching: Arithmetic expressions and precedence: Operators and expression using numeric and relational operators, mixed operands, type conversion, logical operators, bit operations, assignment operator, operator precedence and associativity. Conditional Branching: Applying if and switch statements, nesting if and else, use of break and default with switch case.
- 2. Loops & Functions:** Iteration and loops: use of while, do while and for loops, multiple loop variables, use of break and continue statements. Functions: Introduction, types of functions, functions with array, passing parameters to functions, call by value, call by reference, recursive.
- 3. Arrays & Basic Algorithms:** Array notation and representation, manipulating array elements, using multi-dimensional arrays. Character arrays and strings, Structure, union, enumerated data types, Array of structures, passing arrays to functions. Basic Algorithms: Searching & Basic Sorting Algorithms (Bubble, Insertion and Selection), Finding roots of equations, Notion of order of complexity.
- 4. Pointer & File Handling:** Pointers: Introduction, declaration, applications, Introduction to dynamic memory allocation (malloc, calloc, realloc, free), Use of pointers in self-referential structures, notion of linked list (no implementation) File handling: File I/O functions, Standard C pre-processors, defining and calling macros, command-line arguments.

Course Outcomes (CO): At the end of this course students will be able to:

CO1: To develop simple algorithms for arithmetic and logical problems.

CO2: To translate the algorithms to programs & execution (in C language).

CO3: To implement conditional branching, iteration and recursion.

CO4: To decompose a problem into functions and synthesize a complete program using divide and conquer approach.

CO5: To use arrays, pointers and structures to develop algorithms and programs.

Text Books:

1. The C programming by Kernighan Brian W. and Ritchie Dennis M., Pearson Education.

2. Computer Basics and C Programming by V.Rajaraman , PHI Learning Pvt. Limited, 2015.
3. Computer Concepts and Programming in C, E Balaguruswami, McGraw Hill
4. Computer Science- A Structured Programming Approach Using C, by Behrouz A. Forouzan, Richard F. Gilberg, Thomson, Third Edition , Cengage Learning - 2007.
5. Let Us C By Yashwant P. Kanetkar,BPB Publication.
6. Problem Solving and Program Design in C, by Jeri R. Hanly, Elliot B. Koffman, Pearson Addison-Wesley, 2006.
7. Programming in C by Kochan Stephen G. Pearson Education – 2015.
8. Computer Concepts and Programming by Anami, Angadi and Manvi, PHI Publication.

Programming for Problem Solving Lab (CSE151)

Credit: 0+ 1

Note: A minimum of twenty experiments from the following should be performed.

1. Write a program to calculate the area of triangle using formula $at=\sqrt{s(s-a)(s-b)(s-c)}$
2. Basic salary of an employee is input through the keyboard. The DA is 25% of the basic salary while the HRA is 15% of the basic salary. Provident Fund is deducted at the rate of 10% of the gross salary (BS+DA+HRA).
3. Write a program to determine the roots of quadratic equation.
4. Write a program to find the largest of three numbers using nested if else.
5. Write a program to receive marks of physics, chemistry & maths from user & check its eligibility for course if
 - a. Marks of physics > 40
 - b. Marks of chemistry > 50
 - c. Marks of math's > 60
 - d. Total of physics & math's marks > 150 or
 - e. Total of three subjects marks > 200
6. Write a program to find the value of y for a particular value of n. The a, x, b, n is input by user
 - a. if $n=1$ $y=ax\%b$ if $n=2$ $y=ax^2+b^2$ if $n=3$ $y=a-bx$
 - b. if $n=4$ $y=a+x/b$
7. Write a program to construct a Fibonacci series up to n term.
8. Write a program to find whether the number is Armstrong number.
9. Write a program to generate sum of series $1!+2!+3!+ \dots + n!$
10. Write a program to find the sum of following series $1-X1/1!+X2/2!+\dots+Xn/n!$.
11. Write a program to print the entire prime no between 1 and 300.
12. Write a program to print out all the Armstrong number between 100 and 500.
13. Write a program to draw the following figure:

```
3 2 1
2 1
1

*
**
***
```
14. Write a program to receive a five-digit no and display as like 24689:
 - a. 2
 - b. 4
 - c. 6
 - d. 8
 - e. 9
15. Write a function that return sum of all the odd digits of a given positive no entered through keyboard.
16. Write a program to print area of rectangle using function & return its value to main function.
17. Write a program to calculate the factorial for given number using function.
18. Write a program to find sum of Fibonacci series using function.
19. Write factorial function & use the function to find the sum of series $S=1!+2!+ \dots + n!$.
20. Write a program to find the factorial of given number using recursion.
21. Write a program to find the sum of digits of a 5 digit number using recursion.
22. Write a program to calculate the GCD of given numbers using recursion.

23. Write a program to convert decimal number in to binary number.
24. Write a program to convert binary number into decimal number.
25. Write a program to delete duplicate element in a list of 10 elements & display it on screen.
26. Write a program to merge two sorted array & no element is repeated during merging.
27. Write a program to evaluate the addition of diagonal elements of two square matrixes.
28. Write a program to find the transpose of a given matrix & check whether it is symmetric or not.
29. Write a program to print the multiplication of two N*N (Square) matrix.
30. Write a program in C to check whether the given string is a palindrome or not.
31. Write program to sort the array of character (String) in alphabetical order like STRING in GINRST.
32. Write a program to remove all the blank space from the string & print it, also count the no of characters.
33. Write a program to store the following string "zero", "one" "five". Print the no in words, given in figure as 3205.
34. Write a program to compare two given dates. To store a date uses a structure that contains three members namely day, month and year. If the dates are equal, then display message equal otherwise unequal.
35. Define a structure that can describe a hotel. It should have the member that includes the name, address, grade, room charge and number of rooms. Write a function to print out hotel of given grade in order of room charges.
36. Define a structure called cricket with player name, team name, batting average, for 50 players & 5 teams. Print team wise list contains names of player with their batting average.
37. Write a c program to copy & count the character content of one file says a.txt to another file b.txt.
38. Write a program to take 10 integers from file and write square of these integer in another file.
39. Write a program to read number from file and then write all 'odd' number to file ODD.txt & all even to file EVEN.txt.
40. Write a program to print all the prime number, between 1 to 100 in file prime.txt.
41. Write the following C program using pointer:
 - a. To sort the list of numbers through pointer
 - b. To reverse the string through pointer.
42. Write a program to find the largest no among 20 integers array using dynamic memory allocation.
43. Using Dynamic Memory Allocation, write a program to find the transpose of given matrix.
44. Write a program to find the factorial of given number using command line argument.
45. Write a program to find the sum of digits of a 5-digit number using command line argument.
46. Write a program to implement Singly Linked List.
47. Write a program to implement Stack using dynamic memory allocation.
48. Write a program to implement Queue using dynamic memory allocation.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Mathematics for Machine Learning (CSE201)

Credit: 3+0

Course Objective: Students will be able to learn the fundamental of mathematical concepts used in the area of machine Learning.

Syllabus

1. Introduction to Machine Learning, Probability Theory, Model Selection, The Curse of Dimensionality, Decision Theory, Information Theory.
2. Probability Distributions: Binary Variables, Multinomial Variables, The Gaussian distribution, The Exponential Family, and Nonparametric Methods.
3. Linear Models for Regression: Linear Basis Function Models, The Bias-Variance Decomposition, Bayesian Linear Regression, Bayesian Model Comparison, The Evidence Approximation, Limitations of Fixed Basis Functions
4. Neural Networks and Kernel Methods

Course Outcomes (CO): At the end of this course students will be able to:

CO1: To develop simple algorithms for arithmetic and logical problems.

CO2: To translate the algorithms to programs & execution (in C language).

CO3: To implement conditional branching, iteration and recursion.

CO4: To decompose a problem into functions and synthesize a complete program using divide and conquer approach.

CO5: To use arrays, pointers and structures to develop algorithms and programs.

Text Books:

1. Christopher M. Bishop. 2006. Pattern Recognition and Machine Learning (Information Science and Statistics). Springer-Verlag New York, Inc., Secaucus, NJ, USA.
2. Marc Peter Deisenroth, A. Aldo Faisal, Cheng Soon Ong, 2020, Mathematics For Machine Learning. MIT-Press, Link: <https://mml-book.com>

Mathematics for Machine Learning Lab (CSE251)

Credit: 0+1

1. Implementation of Manhattan and Euclidean Distance in Python.
2. Implementation of Jaccard Index in Python for given dataset.
3. Implementation of Cosine Similarity in Python for given dataset.
4. Apply linear regression on given dataset.
5. Apply multiple regression on given dataset.
6. Implement "AND" gate using McCulloch-Pitts network.
7. Implement "OR" gate using McCulloch-Pitts network.
8. Implement "NOT" gate using McCulloch-Pitts network.
9. Implement "NOR" gate using McCulloch-Pitts network.
10. Implement "NAND" gate using McCulloch-Pitts network.
11. Implement "XOR" gate using McCulloch-Pitts network.
12. Implement "AND" gate using Hebb training algorithm.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

DATA STRUCTURE (CSE202)

Credit: 4+0

Course Objective: To provide the knowledge of basic data structures and its implementations in context of writing efficient programs.

Syllabus

- 1. Introduction:** Basic Terminology, Elementary Data Organization, Built in Data Types in C. Algorithm, Efficiency of an Algorithm, Time and Space Complexity, Asymptotic notations: Big Oh, Big Theta and Big Omega, Time-Space trade-off. Abstract Data Types (ADT) Arrays: Definition, Single and Multidimensional Arrays, Representation of Arrays: Row Major Order, and Column Major Order, Derivation of Index Formulae for 1-D, 2-D, 3-D and n-D Array Application of arrays, Sparse Matrices and their representations. Linked lists: Array Implementation and Pointer Implementation of Singly Linked Lists, Doubly Linked List, Circularly Linked List, Operations on a Linked List. Insertion, Deletion, Traversal, Polynomial Representation and Addition Subtraction & Multiplications of Single variable & two variables Polynomial.
- 2. Stacks and Queues:** Abstract Data Type, Primitive Stack operations: Push & Pop, Array and Linked Implementation of Stack in C, Application of stack: Prefix and Postfix Expressions, Evaluation of postfix expression, Iteration and Recursion- Principles of recursion, Tail recursion, Removal of recursion Problem solving using iteration and recursion with examples such as binary search, Fibonacci numbers, and Hanoi towers. Tradeoffs between iteration and recursion. Queues: Operations on Queue: Create, Add, Delete, Full and Empty, Circular queues, Array and linked implementation of queues in C, Dequeue and Priority Queue.
- 3. Searching & Sorting:** Concept of Searching, Sequential search, Index Sequential Search, Binary Search. Concept of Hashing & Collision resolution Techniques used in Hashing. Sorting: Insertion Sort, Selection, Bubble Sort, Quick Sort, Merge Sort, Heap Sort and Radix Sort.
- 4. Trees & Graphs:** Basic terminology used with Tree, Binary Trees, Binary Tree Representation: Array Representation and Pointer (Linked List) Representation, Binary Search Tree, Strictly Binary Tree, Complete Binary Tree. Extended Binary Trees, Tree Traversal algorithms: Inorder, Preorder and Postorder, Constructing Binary Tree from given Tree Traversal, Operation of Insertion, Deletion, Searching & Modification of data in Binary Search. Threaded Binary trees, Traversing Threaded Binary trees. Huffman coding using Binary Tree. Concept & Basic Operations for AVL Tree, B Tree & Binary Heaps. Graphs: Terminology used with Graph, Data Structure for Graph Representations: Adjacency Matrices, Adjacency List, Adjacency. Graph Traversal: Depth First Search and Breadth First Search, Connected Component, Spanning Trees, Minimum Cost Spanning Trees: Prims and Kruskal algorithm. Transitive Closure and Shortest Path algorithm: Warshal Algorithm and Dijkstra Algorithm.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Describe how arrays, linked lists, stacks, queues, trees, and graphs are represented in memory, used by the algorithms and their common applications.

CO2: Discuss the computational efficiency of the sorting and searching algorithms.

CO3: Implementation of Trees and Graphs and perform various operations on these data structure.

C04: Understanding the concept of recursion, application of recursion and its implementation and removal of recursion.

C05: Identify the alternative implementations of data structures with respect to its performance to solve a real-world problem.

Text Books:

1. Aaron M. Tenenbaum, Yedidyah Langsam and Moshe J. Augenstein, "Data Structures Using C and C++", PHI Learning Private Limited, Delhi India.
2. Horowitz and Sahani, "Fundamentals of Data Structures", Galgotia Publications Pvt Ltd Delhi India.
3. Lipschutz, "Data Structures" Schaum's Outline Series, Tata McGraw-hill Education (India) Pvt. Ltd.
4. Thareja, "Data Structure Using C" Oxford Higher Education.
5. AK Sharma, "Data Structure Using C", Pearson Education India.
6. Rajesh K. Shukla, "Data Structure Using C and C++" Wiley Dreamtech Publication.
7. Michael T. Goodrich, Roberto Tamassia, David M. Mount "Data Structures and Algorithms in C++", Wiley India.
8. P. S. Deshpandey, "C and Data structure", Wiley Dreamtech Publication.
9. R. Kruse et al, "Data Structures and Program Design in C", Pearson Education.
10. Berztiss, AT: Data structures, Theory and Practice, Academic Press.
11. Jean Paul Trembley and Paul G. Sorenson, "An Introduction to Data Structures with applications", McGraw Hill.
12. Adam Drozdek "Data Structures and Algorithm in Java", Cengage Learning.

Data Structure using C Lab (CSE252)

Credit: 0+1

1. Write C Programs to illustrate the concept of the following:
2. Sorting Algorithms-Non-Recursive.
3. Sorting Algorithms-Recursive.
4. Searching Algorithm.
5. Implementation of Stack using Array.
6. Implementation of Queue using Array.
7. Implementation of Circular Queue using Array.
8. Implementation of Stack using Linked List.
9. Implementation of Queue using Linked List.
10. Implementation of Circular Queue using Linked List.
11. Implementation of Tree Structures, Binary Tree, Tree Traversal, Binary Search Tree, Insertion and Deletion in BST.
12. Graph Implementation, BFS, DFS, Minimum cost spanning tree, shortest path algorithm

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Python with Linux (CSE203)

Credit: 2+0

Course Objectives: This course enables students to solve problems in Python.

Syllabus

1. **Programming Basics and Decision Making** : Key features and applications of Python, Python Editors and Compilers (Interpreters), Using different offline and online Python IDE, Interacting with Python programs, **Data types**: Numeric, Boolean, Strings, Lists, Sets, Tuples, Dictionary; **Variables**: Declaration and initialization; **Other concepts**: Operators, Expressions, Indentation, Comments, Casting, Simple Statements: Taking inputs from user, Displaying outputs; Conditional statements: If...Else. **Introduction to Linux**: Understanding Linux, Installation of Python on Linux
2. **Control Flow and Other Programming Concepts** For Loops, While Loops, Break, Continue; **Array**: Looping Array elements, Array methods; **Functions**: Local and Global Variables, Built-in functions, User defined functions, Declaration of a function, Defining the function, Calling of the function, Functions with arguments, Recursion.
3. **OOP and File Handling**: Classes and objects, attributes and methods, constructors and destructors, inheritance, polymorphism, **Exception Handling**: Try...Except; Management of text files: Type of files, various file operations on text files, creating a text file, opening a file, closing a file, reading a text file, writing into a text file, copying a file to another file.
4. **Advance Concepts**: Problem solving- Use of Python to solve real time problems, How Python helps to research problems, Creating various types of graphs corresponding to any data to show different kinds of results and analysis; **Data Analysis**: Understanding problems of data science and machine learning, Creating codes in Python for various data analysis problems, Other advance programs.

Course Outcomes:

CO1: Examine Python syntax and semantics and be fluent in the use of Python flow control and functions.

CO2: Demonstrate proficiency in handling Strings and File Systems.

CO3: Create, run and manipulate Python Programs using core data structures like Lists, Dictionaries and use Regular Expressions.

CO4: Interpret the concepts of Object-Oriented Programming as used in Python.

CO5: Implement exemplary applications related to Network Programming, Web Services and Databases in Python.

Text Books:

1. Core Python Programming, Wesley J. Chun, Second Edition, Pearson.
2. Think Python, Allen Downey, Green Tea Press

3. Introduction to Python, Kenneth A. Lambert, Cengage
4. Python Programming: A Modern Approach, Vamsi Kurama, Pearson
5. Learning Python, Mark Lutz, O'Really
6. Allen B. Downey, ``Think Python: How to Think Like a Computer Scientist “, 2nd edition, Updated for Python 3, Shroff/O ‘Reilly Publishers, 2016 (<http://greenteapress.com/wp/thinkpython/>)
7. Guido van Rossum and Fred L. Drake Jr, —An Introduction to Python – Revised and updated for Python 3.2, Network Theory Ltd., 2011.
8. John V Guttag, —Introduction to Computation and Programming Using Python “, Revised and expanded Edition, MIT Press, 2013
9. Robert Sedgewick, Kevin Wayne, Robert Dondero, —Introduction to Programming in Python: An Inter-disciplinary Approach, Pearson India Education Services Pvt. Ltd., 2016.
10. Timothy A. Budd, —Exploring Python, Mc-Graw Hill Education (India) Private Ltd., 2015.
11. Kenneth A. Lambert, —Fundamentals of Python: First Programs, CENGAGE Learning, 2012.
12. Charles Dierbach, —Introduction to Computer Science using Python: A Computational ProblemSolving Focus, Wiley India Edition, 2013.
13. Paul Gries, Jennifer Campbell and Jason Montojo, —Practical Programming: An Introduction to Computer Science using Python 3, Second edition, Pragmatic Programmers, LLC, 2013.
14. Christopher Negus, LINUX BIBLE , 10th Edition, Wiley.

Python with Linux Lab (CSE253)

Credit: 0+1

1. Krishna and his five friends have decided to go for an industrial visit by sharing the expenses of the fuel equally. Write a Python program to calculate the amount (in Rs) each of them needs to put in for the complete journey. The program should also display **True**, if the amount to be paid by each person is divisible by 3, otherwise it should display **False**.
Hint: Use the relational operators in print statement.
Assumptions: Assume that mileage of the vehicle, amount per litre of fuel and distance for one way are given.
2. Write a python program to find the best of two test average marks out of three test's marks accepted from the user.
3. Develop a Python program to check whether a given number is palindrome or not and also count the number of occurrences of each digit in the input number.
4. Defined as a function F as $F_n = F_{n-1} + F_{n-2}$. Write a Python program which accepts a value for N (where $N > 0$) as input and pass this value to the function. Display suitable error message if the condition for input value is not followed.
5. Develop a Python program to convert binary to decimal, octal to hexadecimal using functions.
6. Write a Python program that accepts a sentence and find the number of words, digits, uppercase letters and lowercase letters.
7. Write a Python program to implement insertion sort and merge sort using lists.
8. Write a program to convert roman numbers in to integer values using dictionaries.
9. Write a function called isphonenum() to recognize a pattern 415-555-4242 without using regular expression and also write the code to recognize the same pattern using regular expression.
10. Develop a Python program that could search the text in a file for phone numbers (+919900889977) and email addresses (sample@gmail.com).
11. Write a Python program to accept a file name from the user and perform the following operations
 - a. Display the first N line of the file.
 - b. Find the frequency of occurrence of the word accepted from the user in the file.
12. Write a Python program to create a ZIP file of a particular folder which contains several files inside it.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Discrete Mathematics (CSE204)

Credit: 4+0

Course Objective: The objective of this course is to teach students how to think logically and mathematically. The course stresses on mathematical reasoning and describes different ways in which mathematical problems could be solved. There are four thematic areas covered in this course: mathematical reasoning, combinatorial analysis, discrete structures, and mathematical modelling.

Syllabus

- 1. Set Theory:** Introduction, Combination of sets, Multisets, Ordered pairs. Proofs of some general identities on sets. Relations: Definition, Operations on relations, Properties of relations, Composite Relations, Equality of relations, Recursive definition of relation, Order of relations. Functions: Definition, Classification of functions, Operations on functions, recursively defined functions. Growth of Functions. Natural Numbers: Introduction, Mathematical Induction, Variants of Induction, Induction with Nonzero Base cases. Proof Methods, Proof by counter – example, Proof by contradiction.
- 2. Algebraic Structures & Lattices:** Definition, Groups, Subgroups and order, Cyclic Groups, Cosets, Lagrange's theorem, Normal Subgroups, Permutation and Symmetric groups, Group Homomorphisms, Definition and elementary properties of Rings and Fields. Lattices: Definition, Properties of lattices – Bounded, Complemented, Modular and Complete lattice. Boolean Algebra: Introduction, Axioms and Theorems of Boolean algebra, Algebraic manipulation of Boolean expressions. Simplification of Boolean Functions, Karnaugh maps, Logic gates, Digital circuits and Boolean algebra.
- 3. Propositional Logic:** Proposition, well-formed formula, Truth tables, Tautology, Satisfiability, Contradiction, Algebra of proposition, Theory of Inference. Predicate Logic: First order predicate, well-formed formula of predicate, quantifiers, Inference theory of predicate logic.
- 4. Trees:** Definition, Binary tree, Binary tree traversal, binary search tree. Graphs: Definition and terminology, Representation of graphs, Multigraphs, Bipartite graphs, planar graphs, Isomorphism and Homeomorphism of graphs, Euler and Hamiltonian paths, Graph coloring, Recurrence Relation & Generating function: Recursive definition of functions, Recursive algorithms, Method of solving recurrences. Combinatorics: Introduction, Counting Techniques, Pigeonhole Principle

Course Outcome (CO): At the end of course, the student will be able to

CO1: Write an argument using logical notation and determine if the argument is or is not valid.

CO2: Understand the basic principles of sets and operations in sets.

CO3: Demonstrate an understanding of relations and functions and be able to determine their properties.

CO4: Demonstrate different traversal methods for trees and graphs.

CO5: Model problems in Computer Science using graphs and trees.

Text Books:

1. Koshy, Discrete Structures, Elsevier Pub. 2008 Kenneth H. Rosen, Discrete Mathematics and Its Applications, 6/e, McGraw-Hill, 2006.
2. B. Kolman, R.C. Busby, and S.C. Ross, Discrete Mathematical Structures, 5/e, Prentice Hall, 2004.
3. E. Scheinerman, Mathematics: A Discrete Introduction, Brooks/Cole, 2000.
4. R.P. Grimaldi, Discrete and Combinatorial Mathematics, 5/e, Addison Wesley, 2004
5. Liptschutz, Seymour, "Discrete Mathematics", McGraw Hill.
6. Trembley, J.P & R. Manohar, "Discrete Mathematical Structure with Application to Computer Science", McGraw Hill.
7. Deo, Narsingh, "Graph Theory With application to Engineering and Computer.Science.", PHI.
8. Krishnamurthy, V., "Combinatorics Theory & Application", East-West Press Pvt. Ltd., New Delhi

Evolutionary Computing (CSE205)

Credit: 3+0

Course Objective: This course enables students to adopt nature-inspired computing models to solve specific problems.

Syllabus

- 1. Evolutionary Computation:** Foundation of Evolutionary theory, Evolutionary Strategies, Evolutionary programming, Evolutionary Algorithms, Evolutionary Algorithm Case Study.
- 2. Genetic Algorithm:** Genetic Representations, Initial Population, Fitness Function, Selection and Reproduction, Genetic Operators (Selection, Crossover, Mutation), Artificial Immune Systems, Other Algorithms Harmony Search, Honey-Bee Optimization, Memetic Algorithms, Co-evolution, Multi Objective Optimization, Artificial Life, Constraint Handling
- 3. Neural Networks:** Neural Networks: Neuron Models, Neuron Architecture, Mathematical Model of Neural Networks, , Artificial Neural Network Learning Methods and Learning Strategies, Supervised Learning, Unsupervised Learning, Reinforcement Learning, Activation Functions, Pattern Classification and Linear Separability, Single and Multilayer Perceptron Network, Self Organizing Map (Kohonen network), Hopfield Network, Back Propagation Network, Radial Basis function Network
- 4. Collective Systems** Collective Behaviour and Swarm Intelligence, Particle Swarm Optimization and Ant Colony Optimization, Artificial evolution of Competing Systems, Artificial Evolution of cooperation and competition. Recent topics from research papers.

Text Books:

1. Dario Floreano, Claudio Mattiussi, "Bio-Inspired Artificial Intelligence: Theories, Methods and Technologies", MIT Press, 2008.
2. Eberhart, E. and Y. Shi., "Computational Intelligence: Concepts and Implementations", Morgan Kaufmann, San Diego, 2007.
3. D. E. Goldberg, "Genetic algorithms in search, optimization, and machine learning", AddisonWesley, 1989
4. R. C. Ebelhart et al., "Swarm Intelligence", Morgan Kaufmann, 2001. • M. Dorigo and T. Stutzle, "Ant Colony Optimization", A Bradford Book, 2004.
5. Leandro Nunes De Castro, Fernando Jose Von Zuben, "Recent Developments in Biologically Inspired Computing", Idea Group Publishing, 2005.
6. Kenneth DeJong, Evolutionary Computation A Unified Approach, 2006.
7. Gusz Eiben and Jim Smith, Introduction to Evolutionary Computing, 2007.
8. Simon O. Haykin, "Neural Networks and Learning Machines", Third Edition, Prentice Hall, 2008. A.E. Elben and J. E. Smith, "Introduction to Evolutionary Computing", Springer, 2010.

Evolutionary Computing Lab (CSE255)

Credit: 0+1

1. Neural Network Implementation
 - a. Implement Perceptron Network
 - b. Implement Adaline Network
 - c. Implement Madaline Network for XOR Function
 - d. Implement Back Propagation Network for XOR Function using Bipolar Inputs and Binary Targets.
 - e. Implement Kohonen Self-Organizing Feature Map
2. Genetic Algorithm Implementation
 - a. To Maximize $F(X_1, X_2) = 4X_1 + 3X_2$
 - b. To minimize $F(X) = X^2$
 - c. Implementation of Traveling Salesman Problem
 - d. To find the roots of Quadratic Equation

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Computer Organization and Architecture (CSE206)

Credit: 3+0

Course Objective: Conceptualize the basics of organizational and architectural issues of a digital computer like CPU, I/O interface and memory organization.

Syllabus

- 1. Introduction:** Functional units of digital system and their interconnections, buses, bus architecture, types of buses and bus arbitration. Register, bus and memory transfer. Processor organization, general registers organization, stack organization and addressing modes.
- 2. Arithmetic and Logic Unit:** Look ahead carries adders. Multiplication: Signed operand multiplication, Booths algorithm and array multiplier. Division and logic operations. Floating point arithmetic operation, Arithmetic & logic unit design. IEEE Standard for Floating Point Numbers. Control Unit: Instruction types, formats, instruction cycles and sub cycles (fetch and execute etc), micro-operations, execution of a complete instruction. Program Control, Reduced Instruction Set Computer, Pipelining. Hardwire and micro programmed control: micro programme sequencing, concept of horizontal and vertical microprogramming.
- 3. Memory:** Basic concept and hierarchy, semiconductor RAM memories, 2D & 2 1/2D memory organization. ROM memories. Cache memories: concept and design issues & performance, address mapping and replacement Auxiliary memories: magnetic disk, magnetic tape and optical disks Virtual memory: concept implementation.
- 4. Input / Output:** Peripheral devices, I/O interface, I/O ports, Interrupts: interrupt hardware, types of interrupts and exceptions. Modes of Data Transfer: Programmed I/O, interrupt initiated I/O and Direct Memory Access., I/O channels and processors. Serial Communication: Synchronous & asynchronous communication, standard communication interfaces.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Study of the basic structure and operation of a digital computer system.

CO2: Analysis of the design of arithmetic & logic unit and understanding of the fixed point and floating-point arithmetic operations.

CO3: Implementation of control unit techniques and the concept of Pipelining.

CO4: Understanding the hierarchical memory system, cache memories and virtual memory.

CO5: Understanding the different ways of communicating with I/O devices and standard I/O interfaces>

Text Books:

1. Computer System Architecture - M. Mano, Pearson Publication
2. Carl Hamacher, Zvonko Vranesic, Safwat Zaky Computer Organization, McGraw-Hill, Fifth Edition, Reprint 2012
3. John P. Hayes, Computer Architecture and Organization, Tata McGraw Hill, Third Edition, 1998.
4. William Stallings, Computer Organization and Architecture-Designing for Performance, Pearson Education, Seventh edition, 2006.

5. Behrooz Parahami, "Computer Architecture", Oxford University Press, Eighth Impression, 2011.
6. David A. Patterson and John L. Hennessy, "Computer Architecture-A Quantitative Approach", Elsevier, a division of reed India Private Limited, Fifth edition, 2012
7. Structured Computer Organization, Tannenbaum (PHI)

Computer Organization & Architecture Lab (256)

Credit: 0+1

1. Implementing HALF ADDER, FULL ADDER using basic logic gates.
2. Implementing Binary -to -Gray, Gray -to -Binary code conversions.
3. Implementing 3-8 line DECODER.
4. Implementing 4x1 and 8x1 MULTIPLEXERS.
5. Verify the excitation tables of various FLIP-FLOPS.
6. Design of an 8-bit Input/ Output system with four 8-bit Internal Registers.
7. Design of an 8-bit Arithmetic Logic Unit.
8. Design the data path of a computer from its register transfer language description.
9. Design the control unit of a computer using either hardwiring or microprogramming based on its register transfer language description.
10. Implement a simple instruction set computer with a control unit and a data path.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Operating Systems (CSE207)

Credit: 4+0

Course Objective: Student will learn about processes and processor management, concurrency and synchronization, memory management schemes, and secondary storage management, security and protection.

Syllabus

- 1. Introduction:** Operating system and functions, Classification of Operating systems- Batch, Interactive, Time-sharing, Real-Time System, Multiprocessor Systems, Multiuser Systems, Multiprocess Systems, Multithreaded Systems, Operating System Structure- Layered structure, System Components, Operating System services, Reentrant Kernels, Monolithic and Microkernel Systems.
- 2. Concurrent Processes:** Process Concept, Principle of Concurrency, Producer / Consumer Problem, Mutual Exclusion, Critical Section Problem, Dekker's solution, Peterson's solution, Semaphores, Test and Set operation; Classical Problem in Concurrency- Dining Philosopher Problem, Sleeping Barber Problem; Inter Process Communication models and Schemes, Process generation.
- 3. CPU Scheduling & Disk Scheduling:** Scheduling Concepts, Performance Criteria, Process States, Process Transition Diagram, Schedulers, Process Control Block (PCB), Process address space, Process identification information, Threads and their management, Scheduling Algorithms, Multiprocessor Scheduling. Deadlock: System model, Deadlock characterization, Prevention, Avoidance and detection, Recovery from deadlock. Disk Scheduling: I/O devices, and I/O subsystems, I/O buffering, Disk storage and disk scheduling.
- 4. Memory Management:** Basic bare machine, Resident monitor, Multiprogramming with fixed partitions, Multiprogramming with variable partitions, Protection schemes, Paging, Segmentation, Paged segmentation, Virtual memory concepts, Demand paging, Performance of demand paging, Page replacement algorithms, Thrashing, Cache memory organization, Locality of reference.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Understand the structure and functions of OS.

CO2: Learn about Processes, Threads and Scheduling algorithms.

CO3: Understand the principles of concurrency and Deadlocks.

CO4: Learn various memory management schemes.

CO5: Study I/O management and File systems.

Text Books:

1. Silberschatz, Galvin and Gagne, "Operating Systems Concepts", Wiley.
2. Sibsankar Halder and Alex A Aravind, "Operating Systems", Pearson Education.
3. Harvey M Dietel, "An Introduction to Operating System", Pearson Education.
4. D M Dhamdhare, "Operating Systems: A Concept based Approach", 2nd Edition, TMH.
5. William Stallings, "Operating Systems: Internals and Design Principles", 6th Edition, Pearson Education

Operating System Lab (CSE257)

Credit: 0+1

1. Study of hardware and software requirements of different operating systems (UNIX, LINUX, WINDOWS XP/ WINDOWS 7/8/10)
2. Execute various UNIX system calls for
 - a. Process management
 - b. File management
 - c. Input/output Systems calls
3. Implement CPU Scheduling Policies:
 - a. SJF
 - b. Priority
 - c. FCFS
 - d. Multi-level Queue
4. Implement file storage allocation technique:
 - a. Contiguous (using array)
 - b. Linked -list (using linked-list)
 - c. Indirect allocation (indexing)
5. Implementation of contiguous allocation techniques:
 - a. Worst-Fit
 - b. Best- Fit
 - c. First- Fit
6. Calculation of external and internal fragmentation
 - a. Free space list of blocks from system
 - b. List process file from the system
7. Implementation of compaction for the continually changing memory layout and calculate total movement of data.
8. Implementation of resource allocation graph (RAG).
9. Implementation of Banker's algorithm.
10. Conversion of resource allocation graph (RAG) to wait for graph (WFG) for each type of method used for storing graph.
11. Implement the solution for Bounded Buffer (producer-consumer) problem using inter process communication techniques-Semaphores.
12. Implement the solutions for Readers-Writer's problem using inter process communication technique -Semaphore.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Software Engineering (CSE208)

Credit: 3+0

Course Objective:

- To provide the idea of decomposing the given problem into Analysis, Design, Implementation, Testing and Maintenance phases.
- To provide an idea of using various process models in the software industry according to given circumstances.
- To gain the knowledge of how Analysis, Design, Implementation, Testing and Maintenance processes are conducted in a software project

Syllabus

- 1. Introduction:** Introduction to Software Engineering, Software Components, Software Characteristics, Software Crisis, Software Engineering Processes, Similarity and Differences from Conventional Engineering Processes, Software Quality Attributes. Software Development Life Cycle (SDLC) Models: Water Fall Model, Prototype Model, Spiral Model, Evolutionary Development Models, Iterative Enhancement Models. Software Requirement Specifications (SRS): Requirement Engineering Process: Elicitation, Analysis, Documentation, Review and Management of User Needs, Feasibility Study, Information Modelling, Data Flow Diagrams, Entity Relationship Diagrams, Decision Tables, SRS Document, IEEE Standards for SRS. Software Quality Assurance (SQA): Verification and Validation, SQA Plans, Software Quality Frameworks, ISO 9000 Models, SEI-CMM Model.
- 2. Software Design:** Basic Concept of Software Design, Architectural Design, Low Level Design: Modularization, Design Structure Charts, Pseudo Codes, Flow Charts, Coupling and Cohesion Measures, Design Strategies: Function Oriented Design, Object Oriented Design, Top-Down and Bottom-Up Design. Software Measurement and Metrics: Various Size Oriented Measures: Halstead's Software Science, Function Point (FP) Based Measures, Cyclomatic Complexity Measures: Control Flow Graphs.
- 3. Software Testing:** Testing Objectives, Unit Testing, Integration Testing, Acceptance Testing, Regression Testing, Testing for Functionality and Testing for Performance, Top Down and Bottom-Up Testing Strategies: Test Drivers and Test Stubs, Structural Testing (White Box Testing), Functional Testing (Black Box Testing), Test Data Suit Preparation, Alpha and Beta Testing of Products. Static Testing Strategies: Formal Technical Reviews (Peer Reviews), Walk Through, Code Inspection, Compliance with Design and Coding Standards.
- 4. Software Maintenance and Software Project Management:** Software as an Evolutionary Entity, Need for Maintenance, Categories of Maintenance: Preventive, Corrective and Perfective Maintenance, Cost of Maintenance, Software Re- Engineering, Reverse Engineering. Software Configuration Management Activities, Change Control Process, Software Version Control, An Overview of CASE Tools. Estimation of Various Parameters such as Cost, Efforts,

Course Outcome (CO): At the end of course, the student will be able to

C01: Explain various software characteristics and analyse different software Development Models.

C02: Demonstrate the contents of a SRS and apply basic software quality assurance practices to ensure that design, development meet or exceed applicable standards.

C03: Compare and contrast various methods for software design

C04: Formulate testing strategy for software systems, employ techniques such as unit testing, Test driven development and functional testing.

C05: Manage software development process independently as well as in teams and make use of Various software management tools for development, maintenance and analysis.

Text Books:

1. RS Pressman, Software Engineering: A Practitioners Approach, McGraw Hill.
2. Pankaj Jalote, Software Engineering, Wiley
3. Rajib Mall, Fundamentals of Software Engineering, PHI Publication.
4. KK Aggarwal and Yogesh Singh, Software Engineering, New Age International Publishers.
5. Ghezzi, M. Jarayeri, D. Manodrioli, Fundamentals of Software Engineering, PHI Publication.
6. Ian Sommerville, Software Engineering, Addison Wesley.
7. Kassem Saleh, "Software Engineering", Cengage Learning.
8. P fleeger, Software Engineering, Macmillan Publication

Software Engineering Lab (CSE258)

Credit: 0+1

Do the following 8 exercises for any two projects given in the list of sample projects or any other projects:

1. Development of problem statement.
2. Preparation of Software Requirement Specification Document, Design Documents.
3. Testing Phase related documents.
4. Preparation of Software Configuration Management and Risk Management related documents.
5. Study and usage of any Design phase CASE tool.
6. Performing the Design by using any Design phase CASE tools.
7. Develop test cases for unit testing and integration testing.
8. Develop test cases for various white box and black box testing techniques.

Sample Projects (Other Projects can be also assigned):

- a.* Passport Automation System
- b.* Book Bank
- c.* Online Exam Registration
- d.* Stock Maintenance System
- e.* Online course reservation system
- f.* E-ticketing
- g.* Software Personnel Management System
- h.* Credit Card Processing
- i.* E-book management System.
- j.* Recruitment system

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Theory of Automata and Formal Languages (CSE209)

Credit: 4+0

Course Objective: This course focuses on the basic theory of Computer Science and formal methods of computation like automata theory, formal languages, grammars and Turing Machines. The objective of this course is to explore the theoretical foundations of computer science from the perspective of formal languages and classify machines by their power to recognize languages.

Syllabus

- 1. Basic Concepts and Automata Theory:** Introduction to Theory of Computation- Automata, Computability and Complexity, Alphabet, Symbol, String, Formal Languages, Deterministic Finite Automaton (DFA)- Definition, Representation, Acceptability of a String and Language, Non Deterministic Finite Automaton (NFA), Equivalence of DFA and NFA, NFA with ϵ -Transition, Equivalence of NFA's with and without ϵ -Transition, Finite Automata with output- Moore Machine, Mealy Machine, Equivalence of Moore and Mealy Machine, Minimization of Finite Automata, Myhill-Nerode Theorem, Simulation of DFA and NFA.
- 2. Regular Expressions and Languages:** Regular Expressions, Transition Graph, Kleen's Theorem, Finite Automata and Regular Expression- Arden's theorem, Algebraic Method Using Arden's Theorem, Regular and Non-Regular Languages- Closure properties of Regular Languages, Pigeonhole Principle, Pumping Lemma, Application of Pumping Lemma, Decidability- Decision properties, Finite Automata and Regular Languages, Regular Languages and Computers, Simulation of Transition Graph and Regular language.
- 3. Regular and Non-Regular Grammars:** Context Free Grammar (CFG)-Definition, Derivations, Languages, Derivation Trees and Ambiguity, Regular Grammars-Right Linear and Left Linear grammars, Conversion of FA into CFG and Regular grammar into FA, Simplification of CFG, Normal Forms- Chomsky Normal Form (CNF), Greibach Normal Form (GNF), Chomsky Hierarchy, Programming problems based on the properties of CFGs.
- 4. Push Down Automata and Properties of Context Free Languages:** Nondeterministic Pushdown Automata (NPDA)- Definition, Moves, A Language Accepted by NPDA, Deterministic Pushdown Automata (DPDA) and Deterministic Context free Languages (DCFL), Pushdown Automata for Context Free Languages, Context Free grammars for Pushdown Automata, two stack Pushdown Automata, Pumping Lemma for CFL, Closure properties of CFL, Decision Problems of CFL, Programming problems based on the properties of CFLs. Turing Machines and Recursive Function Theory: Basic Turing Machine Model, Representation of Turing Machines, Language Acceptability of Turing Machines, Techniques for Turing Machine Construction, Modifications of Turing Machine, Turing Machine as Computer of Integer Functions, Universal Turing machine, Linear Bounded Automata, Church's Thesis, Recursive and Recursively Enumerable language, Halting Problem, Post's Correspondence Problem, Introduction to Recursive Function Theory.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Analyse and design finite automata, pushdown automata, Turing machines, formal languages, and grammars.

CO2: Analyse and design, Turing machines, formal languages, and grammars.

CO3: Demonstrate the understanding of key notions, such as algorithm, computability, decidability, and

complexity through problem solving.

C04: Prove the basic results of the Theory of Computation.

C05: State and explain the relevance of the Church-Turing thesis.

Text Books:

1. Introduction to Automata theory, Languages and Computation, J.E.Hopcraft, R.Motwani, and Ullman. 2nd edition, Pearson Education Asia
2. Introduction to languages and the theory of computation, J Martin, 3rd Edition, Tata McGraw Hill
3. Elements and Theory of Computation, C Papadimitrou and C. L. Lewis, PHI
4. Mathematical Foundation of Computer Science, Y.N.Singh, New Age International.

Artificial Intelligence (CSE301)

Credit: 4+0

Course Objective: This course sheds light on the fundamental of Artificial Intelligence and its applications in various areas.

Syllabus

- 1. Introduction:** Definition – Future of Artificial Intelligence – Characteristics of Intelligent Agents– Typical Intelligent Agents – Problem Solving Approach to Typical AI problems.
- 2. Problem Solving Methods:** Problem solving Methods – Search Strategies- Uninformed – Informed – Heuristics – Local Search Algorithms and Optimization Problems – Searching with Partial Observations – Constraint Satisfaction Problems – Constraint Propagation – Backtracking Search – Game Playing – Optimal Decisions in Games – Alpha – Beta Pruning – Stochastic Games.
- 3. Knowledge Representation:** First Order Predicate Logic – Prolog Programming – Unification – Forward Chaining-Backward Chaining – Resolution – Knowledge Representation – Ontological Engineering-Categories and Objects – Events – Mental Events and Mental Objects – Reasoning Systems for Categories – Reasoning with Default Information.
- 4. Software Agents & Applications:** Architecture for Intelligent Agents – Agent communication – Negotiation and Bargaining – Argumentation among Agents – Trust and Reputation in Multi-agent systems. **Applications:** AI applications – Language Models – Information Retrieval- Information Extraction – Natural Language Processing – Machine Translation – Speech Recognition – Robot – Hardware – Perception – Planning – Moving.

Course Outcome (CO): At the end of course, the student will be able

CO1: To understand the basics of the theory and practice of Artificial Intelligence as a discipline and about intelligent agents.

CO2: Understand search techniques and gaming theory.

CO3: The student will learn to apply knowledge representation techniques and problem-solving strategies to common AI applications.

CO4: Student should be aware of techniques used for classification and clustering.

CO5: Student should aware of basics of pattern recognition and steps required for it.

Text Books:

1. S. Russell and P. Norvig, “Artificial Intelligence: A Modern Approach”, Prentice Hall, Third Edition, 2009.
2. Bratko, “Prolog: Programming for Artificial Intelligence”, Fourth edition, Addison-Wesley Educational Publishers Inc., 2011.
3. M. Tim Jones, —Artificial Intelligence: A Systems Approach, Jones and Bartlett Publishers, Inc. First Edition, 2008
4. Nils J. Nilsson, —The Quest for Artificial Intelligence, Cambridge University Press, 2009.
5. William F. Clocksin and Christopher S. Mellish, Programming in Prolog: Using the ISO Standard, Fifth Edition, Springer, 2003.

6. Gerhard Weiss, —Multi Agent Systems, Second Edition, MIT Press, 2013.
7. David L. Poole and Alan K. Mackworth, —Artificial Intelligence: Foundations of Computational Agents, Cambridge University Press, 2010.

Database Management System (CSE302)

Credit: 3+0

Course Objectives:

- To explain basic database concepts, applications, data models, schemas and instances.
- To demonstrate the use of constraints and relational algebra operations.
- To emphasize the importance of normalization in databases.
- To facilitate students in Database design.
- To familiarize issues of concurrency control and transaction management.

Syllabus

- 1. Introduction:** Overview, Database System vs. File System, Database System Concept and Architecture, Data Model Schema and Instances, Data Independence and Database Language and Interfaces, Data Definitions Language, DML, Overall Database Structure. Data Modelling Using the Entity Relationship Model: ER Model Concepts, Notation for ER Diagram, Mapping Constraints, Keys, Concepts of Super Key, Candidate Key, Primary Key, Generalization, Aggregation, Reduction of an ER Diagrams to Tables, Extended ER Model, Relationship of Higher Degree.
- 2. Relational Data Model and Language:** Relational Data Model Concepts, Integrity Constraints, Entity Integrity, Referential Integrity, Keys Constraints, Domain Constraints, Relational Algebra, Relational Calculus, Tuple and Domain Calculus. Introduction on SQL: Characteristics of SQL, Advantage of SQL. SQL Data Type and Literals. Types of SQL Commands. SQL Operators and Their Procedure. Tables, Views and Indexes. Queries and Sub Queries. Aggregate Functions. Insert, Update and Delete Operations, Joins, Unions, Intersection, Minus, Cursors, Triggers, Procedures in SQL/PL SQL.
- 3. Data Base Design & Normalization:** Functional dependencies, normal forms, first, second, 8 third normal forms, BCNF, inclusion dependence, loss less join decompositions, normalization using FD, MVD, and JDs, alternative approaches to database design.
- 4. Transaction Processing Concept:** Transaction System, Testing of Serializability, Serializability of Schedules, Conflict & View Serializable Schedule, Recoverability, Recovery from Transaction Failures, Log Based Recovery, Checkpoints, Deadlock Handling. Distributed Database: Distributed Data Storage, Concurrency Control, Directory System. Concurrency Control Techniques: Concurrency Control, Locking Techniques for Concurrency Control, Time Stamping Protocols for Concurrency Control, Validation Based Protocol, Multiple Granularity, Multi Version Schemes, Recovery with Concurrent Transaction, Case Study of Oracle.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Apply knowledge of database for real life applications.

CO2: Apply query processing techniques to automate the real time problems of databases.

CO3: Identify and solve the redundancy problem in database tables using normalization.

C04: Understand the concepts of transactions, their processing so they will familiar with broad range of database management issues including data integrity, security and recovery.

C05: Design, develop and implement a small database project using database tools.

Text Books:

1. Korth, Silbertz, Sudarshan," Database Concepts", McGraw Hill.
2. Date C J, "An Introduction to Database Systems", Addison Wesley.
3. Elmasri, Navathe, "Fundamentals of Database Systems", Addison Wesley.
4. O'Neil, Databases, Elsevier Pub.
5. Ramakrishnan "Database Management Systems", McGraw Hill.
6. Leon & Leon, "Database Management Systems", Vikas Publishing House.
7. Bipin C. Desai, "An Introduction to Database Systems", Galgotia Publications.
8. Majumdar & Bhattacharya, "Database Management System", TMH.

Database Management Systems Lab (CSE352)

Credit: 0+1

1. Installing oracle/ MYSQL
2. Creating Entity-Relationship Diagram using case tools.
3. Writing SQL statements Using ORACLE /MYSQL:
 - a. Writing basic SQL SELECT statements.
 - b. Restricting and sorting data.
 - c. Displaying data from multiple tables.
 - d. Aggregating data using group function.
 - e. Manipulating data.
 - f. Creating and managing tables.
4. Implementing Normalization
5. Creating cursor
6. Creating procedure and functions
7. Creating packages and triggers
8. Design and implementation of payroll processing system
9. Design and implementation of Library Information System
10. Design and implementation of Student Information System
11. Automatic Backup of Files and Recovery of Files
12. Sample of Mini project (Design & Development of Data and Application) for following:
 - a. Inventory Control System.
 - b. Material Requirement Processing.
 - c. Hospital Management System.
 - d. Railway Reservation System.
 - e. Personal Information System.
 - f. Web Based User Identification System.
 - g. Timetable Management System.
 - h. Hotel Management System

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner

Design and Analysis of Algorithms (CSE303)

Credit: 4+0

Course Objective: The goal of this course is to provide a solid background in the design and analysis of the major classes of algorithms. At the end of the course students will be able to develop their own versions for a given computational task and to compare and contrast their performance.

Syllabus

1. Introduction: Algorithms, Analyzing Algorithms, Complexity of Algorithms, Growth of Functions, Performance Measurements, Sorting and Order Statistics - Shell Sort, Quick Sort, Merge Sort, Heap Sort, Comparison of Sorting Algorithms, Sorting in Linear Time.
2. Advanced Data Structures: Red-Black Trees, B – Trees, Binomial Heaps, Fibonacci Heaps, Tries, Skip List.
3. Divide and Conquer with Examples Such as Sorting, Matrix Multiplication, Convex Hull and Searching. Greedy Methods with Examples Such as Optimal Reliability Allocation, Knapsack, Minimum Spanning Trees – Prim's and Kruskal's Algorithms, Single Source Shortest Paths - Dijkstra's and Bellman Ford Algorithms.
4. Dynamic Programming with Examples Such as Knapsack. All Pair Shortest Paths – Warshal's and Floyd's Algorithms, Resource Allocation Problem. Backtracking, Branch and Bound with Examples Such as Travelling Salesman Problem, Graph Coloring, n-Queen Problem, Hamiltonian Cycles and Sum of Subsets.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Design new algorithms, prove them correct, and analyse their asymptotic and absolute runtime and memory demands.

CO2: Find an algorithm to solve the problem (create) and prove that the algorithm solves the problem correctly (validate).

CO3: Understand the mathematical criterion for deciding whether an algorithm is efficient, and know many practically important problems that do not admit any efficient algorithms.

CO4: Apply classical sorting, searching, optimization and graph algorithms.

CO5: Understand basic techniques for designing algorithms, including the techniques of recursion, divide-and-conquer, and greedy.

Text Books:

1. Thomas H. Cormen, Charles E. Leiserson and Ronald L. Rivest, "Introduction to Algorithms", Printice Hall of India.
2. E. Horowitz & S Sahni, "Fundamentals of Computer Algorithms",

3. Aho, Hopcraft, Ullman, "The Design and Analysis of Computer Algorithms" Pearson Education, 2008.
4. LEE "Design & Analysis of Algorithms (POD)", McGraw Hill
5. Richard E. Neapolitan "Foundations of Algorithms" Jones & Bartlett Learning
6. Jon Kleinberg and Éva Tardos, Algorithm Design, Pearson, 2005.
7. Michael T Goodrich and Roberto Tamassia, Algorithm Design: Foundations, Analysis, and Internet Examples, Second Edition, Wiley, 2006.
8. Harry R. Lewis and Larry Denenberg, Data Structures and Their Algorithms, Harper Collins, 1997
9. Robert Sedgewick and Kevin Wayne, Algorithms, fourth edition, Addison Wesley, 2011.
10. Harsh Bhasin, "Algorithm Design and Analysis", First Edition, Oxford University Press.
11. Gilles Brassard and Paul Bratley, Algorithmics: Theory and Practice, Prentice Hall, 1995.

Design and Analysis of Algorithm Lab (CSE353)

Credit: 0+1

1. Program for Recursive Binary & Linear Search.
2. Program for Heap Sort.
3. Program for Merge Sort.
4. Program for Selection Sort.
5. Program for Insertion Sort.
6. Program for Quick Sort.
7. Knapsack Problem using Greedy Solution
8. Perform Travelling Salesman Problem
9. Find Minimum Spanning Tree using Kruskal's Algorithm
10. Implement N Queen Problem using Backtracking
11. Sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of $n > 5000$ and record the time taken to sort. Plot a graph of the time taken versus non graph sheet. The elements can be read from a file or can be generated using the random number generator. Demonstrate using Java how the divide and- conquer method works along with its time complexity analysis: worst case, average case and best case.
12. Sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of $n > 5000$, and record the time taken to sort. Plot a graph of the time taken versus non graph sheet. The elements can be read from a file or can be generated using the random number generator. Demonstrate how the divide and- conquer method works along with its time complexity analysis: worst case, average case and best case.
13. Implement, the 0/1 Knapsack problem using
 - a. Dynamic Programming method
 - b. Greedy method
14. From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.
15. Find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm. Use Union-Find algorithms in your program.
16. Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.
17. Write programs to
 - a. Implement All-Pairs Shortest Paths problem using Floyd's algorithm.
 - b. Implement Travelling Sales Person problem using Dynamic programming.
18. Design and implement to find a subset of a given set $S = \{S_1, S_2, \dots, S_n\}$ of n positive integers whose SUM is equal to a given positive integer d. For example, if $S = \{1, 2, 5, 6, 8\}$ and $d = 9$, there are two solutions $\{1, 2, 6\}$ and $\{1, 8\}$. Display a suitable message, if the given problem instance doesn't have a solution.
19. Design and implement to find all Hamiltonian Cycles in a connected undirected Graph G of n vertices using backtracking principle.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner. It is also suggested that open source tools should be preferred to conduct the lab (C, C++ etc).

Computer Network (CSE304)

Credit: 4+0

Course Objective: This course studies the standard models for the layered approach to communication between autonomous machines in a network, and the main characteristics of data transmission across various physical link types. It considers how to design networks and protocols for diverse situations, analyses several application and support protocols from a distributed systems viewpoint, and identifies significant problem areas in networked communications.

Syllabus

- 1. Introductory Concepts:** Goals and applications of networks, Categories of networks, Organization of the Internet, ISP, Network structure and architecture (layering principles, services, protocols and standards), The OSI reference model, TCP/IP protocol suite, Network devices and components. Physical Layer: Network topology design, Types of connections, Transmission media, Signal transmission and encoding, Network performance and transmission impairments, switching techniques and multiplexing.
- 2. Link Layer:** Framing, Error Detection and Correction, Flow control (Elementary Data Link Protocols, Sliding Window protocols). Medium Access Control and Local Area Networks: Channel allocation, multiple access protocols, LAN standards, Link layer switches & bridges (learning bridge and spanning tree algorithms).
- 3. Network Layer:** Point-to-point networks, Logical addressing, Basic internetworking (IP, CIDR, ARP, RARP, DHCP, and ICMP), Routing, forwarding and delivery, Static and dynamic routing, Routing algorithms and protocols, Congestion control algorithms, IPv6.
- 4. Transport Layer:** Process-to-process delivery, Transport layer protocols (UDP and TCP), Multiplexing, Connection management, Flow control and retransmission, Window management, TCP Congestion control, Quality of service. **Application Layer:** Domain Name System, World Wide Web and Hyper Text Transfer Protocol, Electronic mail, File Transfer Protocol, Remote login, Network management, Data compression, Cryptography – basic concepts.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Explain basic concepts, OSI reference model, services and role of each layer of OSI model and TCP/IP, networks devices and transmission media, Analog and digital data transmission.

CO2: Apply channel allocation, framing, error and flow control techniques.

CO3: Describe the functions of Network Layer i.e., Logical addressing, subnetting & Routing Mechanism.

CO4: Explain the different Transport Layer function i.e., Port addressing, Connection Management, Error control and Flow control mechanism.

CO5: Explain the functions offered by session and presentation layer and their Implementation.

CO6: Explain the different protocols used at application layer i.e., HTTP, SNMP, SMTP, FTP, TELNET and VPN.

Text Books:

1. Behrouz Forouzan, "Data Communication and Networking", McGraw Hill
2. Andrew Tanenbaum "Computer Networks", Prentice Hall.
3. William Stallings, "Data and Computer Communication", Pearson.
4. Kurose and Ross, "Computer Networking- A Top-Down Approach", Pearson.
5. Peterson and Davie, "Computer Networks: A Systems Approach", Morgan Kaufmann
6. W. A. Shay, "Understanding Communications and Networks", Cengage Learning.
7. D. Comer, "Computer Networks and Internets", Pearson.
8. Behrouz Forouzan, "TCP/IP Protocol Suite", McGraw Hill.

Computer Network Lab (CSE354)

Credit: 0+1

1. Implementation of Stop and Wait Protocol and Sliding Window Protocol.
2. Study of Socket Programming and Client – Server model.
3. Write a code simulating ARP /RARP protocols.
4. Write a code simulating PING and TRACEROUTE commands.
5. Create a socket for HTTP for web page upload and download.
6. Write a program to implement RPC (Remote Procedure Call).
7. Implementation of Subnetting.
8. Applications using TCP Sockets like:
 - a. Echo client and echo server
 - b. Chat
 - c. File Transfer
9. Applications using TCP and UDP Sockets like:
 - a. DNS
 - b. SNMP
 - c. File Transfer
10. Study of Network simulator (NS).and Simulation of Congestion Control Algorithms using NS
11. Perform a case study about the different routing algorithms to select the network path with its optimum and economical during data transfer.
 - a. Link State routing
 - b. Flooding
 - c. Distance vector
12. To learn handling and configuration of networking hardware like RJ-45 connector, CAT-6 cable, crimping tool, etc.
13. Configuration of router, hub, switch etc. (using real devices or simulators).
14. Running and using services/commands like ping, traceroute, nslookup, arp, telnet, ftp, etc.
15. Network packet analysis using tools like Wireshark, tcpdump, etc.
16. Network simulation using tools like Cisco Packet Tracer, NetSim, OMNeT++, NS2, NS3, etc.
17. Socket programming using UDP and TCP (e.g., simple DNS, data & time client/server, echo client/server, iterative & concurrent servers).

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner It is also suggested that open source tools should be preferred to conduct the lab (C , C++ , Java , NS3, Mininet, Opnet, TCP Dump, Wireshark etc.

Compiler Design (CSE305)

Credit: 3+0

Course Objective: The main objective of this course is to introduce the major concept areas of language translation and compiler design and to develop an awareness of the function and complexity of modern compilers. This course is a study of the theory and practice required for the design and implementation of interpreters and compilers for programming languages.

Syllabus

- 1. Introduction to Compiler:** Phases and passes, Bootstrapping, Finite state machines and regular expressions and their applications to lexical analysis, Optimization of DFA-Based Pattern Matchers implementation of lexical analyzers, lexical-analyzer generator, LEX compiler, Formal grammars and their application to syntax analysis, BNF notation, ambiguity, YACC. The syntactic specification of programming languages: Context free grammars, derivation and parse trees, capabilities of CFG.
- 2. Basic Parsing Techniques:** Parsers, Shift reduce parsing, operator precedence parsing, top-down parsing, predictive parsers Automatic Construction of efficient Parsers: LR parsers, the canonical Collection of LR (0) items, constructing SLR parsing tables, constructing Canonical LR parsing tables, Constructing LALR parsing tables, using ambiguous grammars, an automatic parser generator, implementation of LR parsing tables.
- 3. Syntax-directed Translation:** Syntax-directed Translation schemes, Implementation of Syntax-directed Translators, Intermediate code, postfix notation, Parse trees & syntax trees, three address code, quadruple & triples, translation of assignment statements, Boolean expressions, statements that alter the flow of control, postfix translation, translation with a top-down parser. More about translation: Array references in arithmetic expressions, procedures call, declarations and case statements.
- 4. Symbol Tables & Code Generation:** Data structure for symbols tables, representing scope information. Run-Time Administration: Implementation of simple stack allocation scheme, storage allocation in block structured language. Error Detection & Recovery: Lexical Phase errors, syntactic phase errors semantic errors. Code Generation: Design Issues, the Target Language. Addresses in the Target Code, Basic Blocks and Flow Graphs, Optimization of Basic Blocks, Code Generator. Code optimization: Machine-Independent Optimizations, Loop optimization, DAG representation of basic blocks, value numbers and algebraic laws, Global Data-Flow analysis.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Acquire knowledge of different phases and passes of the compiler and also able to use the compiler tools like LEX, YACC, etc. Students will also be able to design different types of compiler tools to meet the requirements of the realistic constraints of compilers.

CO2: Understand the parser and its types i.e., Top-Down and Bottom-up parsers and construction of LL, SLR, CLR, and LALR parsing table.

CO3: Implement the compiler using syntax-directed translation method and get knowledge about the synthesized and inherited attributes.

C04: Acquire knowledge about run time data structure like symbol table organization and different techniques used in that.

C05: Understand the target machine's run time environment, its instruction set for code generation and techniques used for code optimization.

Text Books:

1. Aho, Sethi & Ullman, "Compilers: Principles, Techniques and Tools", Pearson Education
2. K. Muneeswaran, Compiler Design, First Edition, Oxford University Press
3. J.P. Bennet, "Introduction to Compiler Techniques", Second Edition, McGraw-Hill, 2003.
4. Henk Alblas and Albert Nymeyer, "Practice and Principles of Compiler Building with C", PHI, 2001.
5. V Raghvan, "Principles of Compiler Design", McGraw-Hill.
6. Kenneth Loudon, "Compiler Construction", Cengage Learning.
7. Charles Fischer and Ricard LeBlanc, "Crafting a Compiler with C", Pearson Education

Compiler Design Lab (CSE355)

Credit: 0+1

1. Design and implement a lexical analyzer for given language using C and the lexical analyzer should ignore redundant spaces, tabs and new lines.
2. Implementation of Lexical Analyzer using Lex Tool
 - a. Generate YACC specification for a few syntactic categories.
 - b. Program to recognize a valid arithmetic expression that uses operator +, -, * and /.
 - c. Program to recognize a valid variable which starts with a letter followed by any number of letters or digits.
 - d. Implementation of Calculator using LEX and YACC
 - e. Convert the BNF rules into YACC form and write code to generate abstract syntax tree
3. Write program to find ϵ - closure of all states of any given NFA with ϵ transition.
4. Write program to convert NFA with ϵ transition to NFA without ϵ transition.
5. Write program to convert NFA to DFA.
6. Write program to minimize any given DFA.
7. Develop an operator precedence parser for a given language.
8. Write program to find Simulate First and Follow of any given grammar.
9. Construct a recursive descent parser for an expression.
10. Construct a Shift Reduce Parser for a given language.
11. Write a program to perform loop unrolling.
12. Write a program to perform constant propagation.
13. Implement Intermediate code generation for simple expressions.
14. Implement the back end of the compiler which takes the three-address code and produces the 8086 assembly language instructions that can be assembled and run using an 8086 assembler. The target assembly instructions can be simple move, add, sub, jump etc.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner. It is also suggested that open-source tools should be preferred to conduct the lab (C, C++, Lex or Flex and YACC tools (Unix/Linux utilities) etc).

Machine Learning (CSE306)

Credit: 4+0

Course Objectives: This course provides an advanced level of understanding to machine learning and statistical pattern recognition. It offers some of the most cost-effective approaches to automated knowledge acquisition in emerging data-rich disciplines and focuses on the theoretical understanding of these methods, as well as their computational implications

Syllabus

- 1. Introduction:** Learning, Types of Learning, well defined learning problems, Designing a Learning System, History of ML, Introduction of Machine Learning Approaches – (Artificial Neural Network, Clustering, Reinforcement Learning, Decision Tree Learning, Bayesian networks, Support Vector Machine, Genetic Algorithm), Issues in Machine Learning and Data Science Vs Machine Learning.
- 2. Regression and Decision Tree Learning:** Linear Regression and Logistic Regression BAYESIAN LEARNING - Bayes theorem, Concept learning, Bayes Optimal Classifier, Naïve Bayes classifier, Bayesian belief networks, EM algorithm. SUPPORT VECTOR MACHINE: Introduction, Types of support vector kernel – (Linear kernel, polynomial kernel, and Gaussian kernel), Hyperplane – (Decision surface), Properties of SVM, and Issues in SVM. DECISION TREE LEARNING: Decision tree learning algorithm, Inductive bias, Inductive inference with decision trees, Entropy and information theory, Information gain, ID-3 Algorithm, Issues in Decision tree learning. INSTANCE-BASED LEARNING – k-Nearest Neighbour Learning, Locally Weighted Regression, Radial basis function networks, Case-based learning.
- 3. Artificial Neural Networks:** Perceptron's, Multilayer perceptron, Gradient descent and the Delta rule, Multilayer networks, Derivation of Backpropagation Algorithm, Generalization, Unsupervised Learning – SOM Algorithm and its variant; DEEP LEARNING - Introduction, concept of convolutional neural network , Types of layers – (Convolutional Layers , Activation function , pooling , fully connected) , Concept of Convolution (1D and 2D) layers, Training of network, Case study of CNN for e.g. on Diabetic Retinopathy, Building a smart speaker, Self-deriving car etc.
- 4. Reinforcement Learning:** Introduction to Reinforcement Learning, Learning Task, Example of Reinforcement Learning in Practice, Learning Models for Reinforcement – (Markov Decision process, Q Learning - Q Learning function, Q Learning Algorithm), Application of Reinforcement Learning, Introduction to Deep Q Learning. GENETIC ALGORITHMS: Introduction, Components, GA cycle of reproduction, Crossover, Mutation, Genetic Programming, Models of Evolution and Learning, Applications.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Understand the need for machine learning for various problem solving.

CO2: Understand a wide variety of learning algorithms and how to evaluate models generated from data.

CO3: Understand the latest trends in machine learning.

CO4: Design appropriate machine learning algorithms and apply the algorithms to a real-world problem.

C05: Optimize the models learned and report on the expected accuracy that can be achieved by applying the models.

Text Books:

1. Tom M. Mitchell, —Machine Learning, McGraw-Hill Education (India) Private Limited, 2013.
2. Ethem Alpaydin, —Introduction to Machine Learning (Adaptive Computation and Machine Learning), The MIT Press 2004.
3. Stephen Marsland, —Machine Learning: An Algorithmic Perspective, CRC Press, 2009.
4. Bishop, C., Pattern Recognition and Machine Learning. Berlin: Springer-Verlag.

Machine Learning Lab (CSE356)

Credit: 0+1

1. Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSV file.
2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.
3. Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.
4. Build an Artificial Neural Network by implementing the Back propagation algorithm and test the same using appropriate data sets.
5. Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.
6. Assuming a set of documents that need to be classified, use the naïve Bayesian Classifier model to perform this task. Built-in python classes/API can be used to write the program. Calculate the accuracy, precision, and recall for your data set.
7. Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set. You can use Python ML library classes/API.
8. Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same data set for clustering using k-Means algorithm. Compare the results of these two algorithms and comment on the quality of clustering. You can add Python ML library classes/API in the program.
9. Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions. Python ML library classes can be used for this problem.
10. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.
11. Carry out the performance analysis of classification algorithms on a specific dataset.

Note: The Instructor may add/delete/modify/tune experiments, wherever he/she feels in a justified manner.

Syllabus of Professional Elective Courses

Offered by Departments

Elective-1

(Offered by Department)

Data Mining and Data Warehousing (ECSE115)

Credit: 3+0

Course Objective:

- The course content enables students to:
- Analyze the difference between On Line Transaction Processing and On-Line analytical processing.
- Create Multidimensional schemas suitable for data warehousing.
- Understand various data mining functionalities.
- Understand in detail about data mining algorithms.

Syllabus

1. **Data Warehousing and Business Analysis:** Data warehousing Components –Building a Data warehouse –Data Warehouse Architecture – DBMS Schemas for Decision Support – Data Extraction, Cleanup, and Transformation Tools –Metadata – reporting – Query tools and Applications – Online Analytical Processing (OLAP) – OLAP and Multidimensional Data Analysis.
2. **Data Mining:** Data Mining Functionalities – Data Preprocessing – Data Cleaning – Data Integration and Transformation – Data Reduction – Data Discretization and Concept Hierarchy Generation- Architecture of A Typical Data Mining Systems- Classification of Data Mining Systems. **Association Rule Mining:** - Efficient and Scalable Frequent Item set Mining Methods – Mining Various Kinds of Association Rules – Association Mining to Correlation Analysis – Constraint-Based Association Mining.
3. **Classification and Prediction:** Issues Regarding Classification and Prediction – Classification by Decision Tree Introduction – Bayesian Classification – Rule Based Classification – Classification by Back propagation – Support Vector Machines – Associative Classification – Lazy Learners – Other Classification Methods – Prediction – Accuracy and Error Measures – Evaluating the Accuracy of a Classifier or Predictor – Ensemble Methods – Model Section.
4. **Cluster Analysis, Mining Object, Spatial, Multimedia, Text and Web Data:** Types of Data in Cluster Analysis – A Categorization of Major Clustering Methods – Partitioning Methods – Hierarchical methods – Density-Based Methods – Grid-Based Methods – Model-Based Clustering Methods – Clustering High-Dimensional Data – Constraint-Based Cluster Analysis – Outlier Analysis. **Mining Object, Spatial, Multimedia, Text and Web Data:** Multidimensional Analysis and Descriptive Mining of Complex Data Objects – Spatial Data Mining – Multimedia Data Mining – Text Mining – Mining the World Wide Web.

Course Outcome (CO): At the end of course, the student will be able to

CO1: To understand the principles of Data Warehousing and Data Mining.

CO2: To be familiar with the Data warehouse architecture and its Implementation.

CO3: To know the Architecture of a Data Mining system.

CO4: To understand the various Data preprocessing Methods.

CO5: To perform classification and prediction of data.

Text Books:

1. Jiawei Han, Micheline Kamber and Jian Pei “Data Mining Concepts and Techniques”, Third Edition, Elsevier, 2011.
2. Alex Berson and Stephen J. Smith “Data Warehousing, Data Mining & OLAP”, Tata McGraw – Hill Edition, Tenth Reprint 2007.

3. K.P. Soman, Shyam Diwakar and V. Ajay "Insight into Data mining Theory and Practice", Easter Economy Edition, Prentice Hall of India, 2006.
4. G. K. Gupta "Introduction to Data Mining with Case Studies", Easter Economy Edition, Prentice Hall of India, 2006.
5. Pang-Ning Tan, Michael Steinbach and Vipin Kumar "Introduction to Data Mining", Pearson.

Advanced Database Management System (ECSE116)

Credit: 3+0

Course Objectives:

- To provide a strong foundation in advanced database concepts from an industry perspective.
- To covers advanced data modeling concepts like OOD Modeling and ORD Modeling
- To learn query processing and transaction management concepts for object-relational database and distributed database

Syllabus

1. Transaction and schedules, Concurrent Execution of transaction, Conflict and View Serializability, Testing for Serializability, Concepts in Recoverable and Cascade less schedules. Lock based protocols, time stamp-based protocols, Multiple Granularity and Multiversion Techniques, enforcing serializability by Locks, Locking system with multiple lock modes, architecture for Locking scheduler
2. Distributed Transactions Management, Data Distribution, fragmentation and Replication Techniques, Distributed Commit, Distributed Locking schemes, Long duration transactions, Two Phase Commit protocol.
3. Issues of Recovery and atomicity in Distributed Databases, Traditional recovery techniques, Log based recovery, Recovery with Concurrent Transactions, Recovery in Message passing systems, Checkpoints, Algorithms for recovery line, Concepts in Orphan and Inconsistent Messages.
4. Distributed Query Processing, Multiway Joins, Semi joins, Cost based query optimization for distributed database, Updating replicated data, protocols for Distributed Deadlock Detection, Eager and Lazy Replication Techniques

Course Outcome (CO): At the end of course, the student will be able

CO1: To understand the principles of transaction and schedules.

CO2: To be familiar with the concepts of distributed transaction management.

CO3: To know the issues of recovery in transactions.

CO4: To understand the issues of the concurrent transaction processing.

CO5: To know about distributed query processing.

Text Books:

1. Silberschatz, Korth and Sudershan, Database System Concept', Mc Graw Hill
2. Ramakrishna and Gehrke, ' Database Management System, Mc Graw Hill
3. Garcia-Molina, Ullman, Widom, ' Database System Implementation' Pearson Education 2. Ceei and Pelagatti, 'Distributed Database', TMH
4. Singhal and Shivratri, 'Advance Concepts in Operating Systems' MC Graw Hill

Design and Analysis of Parallel Algorithms (ECSE117)

Credit: 3+0

Course Objective: This module introduces theoretical design principles and analysis techniques that enable the creation and evaluation of efficient, scalable and portable algorithms for parallel computers. Concrete examples will span a range of application areas and architectural models seeking wherever possible to exploit commonality through appropriate abstraction.

Syllabus

1. **Introduction:** Sequential model, need of alternative model, parallel computational models such as PRAM, LMCC, Hypercube, Cube Connected Cycle, Butterfly, Perfect Shuffle Computers, Tree model, Pyramid model, Fully Connected model, PRAM-CREW, EREW models, simulation of one model from another one.
2. **Performance Measures of Parallel Algorithms:** speed-up and efficiency of PA, Cost-optimality, an example of illustrate Cost- optimal algorithms- such as summation, Min/Max on various models. **Parallel Sorting Networks:** Parallel Merging Algorithms on CREW/EREW/MCC, Parallel Sorting Networks CREW/EREW/MCC/, linear array.
3. **Parallel Searching Algorithm:** Kth element, Kth element in X+Y on PRAM, Parallel Matrix Transportation and Multiplication Algorithm on PRAM, MCC, Vector-Matrix Multiplication, Solution of Linear Equation, Root finding.
4. **Graph Algorithms:** Connected Graphs, search and traversal, Combinatorial Algorithms- Permutation, Combinations, Derrangements.

Course Outcome (CO): At the end of course, the student will be able

CO1: To know about the models of parallel computing.

CO2: To be familiar with the parallel sorting networks.

CO3: To know about parallel searching algorithms.

CO4: To understand the issues of graph algorithms in parallel algorithms.

Text Books:

1. M.J. Quinn, "Designing Efficient Algorithms for Parallel Computer", McGrawHill.
2. S.G. Akl, "Design and Analysis of Parallel Algorithms", by Academic Press
3. S.G. Akl, "Parallel Sorting Algorithm" by Academic Press

Computational Intelligence (ECSE118)

Credit: 3+0

Course Objectives:

The course goal is to make students familiar with basic principles of various computational methods of data processing that can commonly be called computational intelligence (CI). Examples of CI are nature-inspired methods (neural nets, evolutionary algorithms), fuzzy systems, as well as various probabilistic methods under uncertainty (e.g. Bayesian models) and machine learning methods (e.g. reinforcement learning). After the course the students will be able to conceptually understand the important terms and algorithms of CI, such that they would be able to choose appropriate method(s) for a given task.

Syllabus

- 1. Introduction:** Introduction to Artificial Intelligence-Search-Heuristic Search-A* algorithm-Game Playing- Alpha-Beta Pruning-Expert Systems-Inference-Rules-Forward Chaining and Backward Chaining- Genetic Algorithms.
- 2. Knowledge Representation and Reasoning:** Proposition Logic - First Order Predicate Logic - Unification - Forward Chaining -Backward Chaining - Resolution - Knowledge Representation - Ontological Engineering - Categories and Objects - Events - Mental Events and Mental Objects - Reasoning Systems for Categories - Reasoning with Default Information - Prolog Programming. **UNCERTAINTY:** Non monotonic reasoning-Fuzzy Logic-Fuzzy rules-fuzzy inference-Temporal Logic-Temporal Reasoning-Neural Networks-Neuro-fuzzy Inference.
- 3. Learning:** Probability basics - Bayes Rule and its Applications - Bayesian Networks - Exact and Approximate Inference in Bayesian Networks - Hidden Markov Models - Forms of Learning - Supervised Learning - Learning Decision Trees - Regression and Classification with Linear Models - Artificial Neural Networks - Nonparametric Models - Support Vector Machines - Statistical Learning - Learning with Complete Data - Learning with Hidden Variables- The EM Algorithm - Reinforcement Learning
- 4. Intelligence and Applications:** Natural language processing-Morphological Analysis-Syntax Analysis-Semantic Analysis-All applications - Language Models - Information Retrieval - Information Extraction - Machine Translation - Machine Learning - Symbol-Based - Machine Learning: Connectionist - Machine Learning.

Course Outcome (CO): Upon completion of the course, the students will be able to

CO1: Provide a basic exposition to the goals and methods of Computational Intelligence.

CO2: Study of the design of intelligent computational techniques.

CO3: Apply the Intelligent techniques for problem solving.

CO4: Improve problem solving skills using the acquired knowledge in the areas of, reasoning, natural language understanding, computer vision, automatic programming and machine learning.

Text Books:

1. Stuart Russell, Peter Norvig, —Artificial Intelligence: A Modern Approach, Third Edition, Pearson Education / Prentice Hall of India, 2010.
2. Elaine Rich and Kevin Knight, —Artificial Intelligence, Third Edition, Tata McGraw-Hill, 2010.
3. Patrick H. Winston. "Artificial Intelligence", Third edition, Pearson Edition, 2006.
4. Dan W.Patterson, —Introduction to Artificial Intelligence and Expert Systems, PHI, 2006.
5. Nils J. Nilsson, —Artificial Intelligence: A new Synthesis, Harcourt Asia Pvt. Ltd., 2000.

Parallel and Distributed Computing (ECSE119)

Credit: 3+0

Course Objective: To learn the advanced concepts of Parallel and Distributed Computing and its implementation for assessment of understanding the course by the students.

Syllabus

- 1. Parallelism Fundamentals and Parallel Architecture:** Scope and issues of parallel and distributed computing, Parallelism, Goals of parallelism, Parallelism and concurrency, Multiple simultaneous computations, Programming Constructs for creating Parallelism, communication, and coordination. Programming errors not found in sequential programming like data races, higher level races, lack of liveness. **Parallel Architecture:** Architecture of Parallel Computer, Communication Costs, parallel computer structure, architectural classification schemes, Multicore processors, Memory Issues: Shared vs. distributed, Symmetric multiprocessing (SMP), SIMD, vector processing, GPU, coprocessing, Flynn's Taxonomy, Instruction Level support for parallel programming, Multiprocessor caches and Cache Coherence, Non-Uniform Memory Access (NUMA).
- 2. Parallel Decomposition and Parallel Performance:** Need for communication and coordination/synchronization, Scheduling and contention, Independence and partitioning, Task- Based Decomposition, Data Parallel Decomposition, Actors and Reactive Processes, Load balancing, Data Management, Impact of composing multiple concurrent components, Power usage and management. Sources of Overhead in Parallel Programs, Performance metrics for parallel algorithm implementations, Performance measurement, The Effect of Granularity on Performance Power Use and Management, Cost-Performance trade-off;
- 3. Distributed Computing:** Definition, Relation to parallel systems, synchronous vs asynchronous execution, design issues and challenges, A Model of Distributed Computations, A Model of distributed executions, Models of communication networks, Global state of distributed system, Models of process communication. Communication and Coordination: Shared Memory, Consistency, Atomicity, Message- Passing, Consensus, Conditional Actions, Critical Paths, Scalability, cache coherence in multiprocessor systems, synchronization mechanism. **CUDA programming model:** Overview of CUDA, Isolating data to be used by parallelized code, API function to allocate memory on the parallel computing device, API function to transfer data to parallel computing device, Concepts of Threads, Blocks, Grids, developing kernel function that will be executed by threads in the parallelized part, Launching the execution of kernel function by parallel threads, transferring data back to host processor with API function call.
- 4. Parallel Algorithms design, Analysis, and Programming:** Parallel Algorithms, Parallel Graph Algorithms, Parallel Matrix Computations, Critical paths, work and span and relation to Amdahl's law, Speed-up and scalability, naturally parallel algorithms, Parallel algorithmic patterns like divide and conquer, map and reduce, Specific algorithms like parallel Merge Sort, Parallel graph algorithms, parallel shortest path, parallel spanning tree, Producer-consumer and pipelined algorithms.

Course outcomes (CO): On completion of this course, the students will be able to

CO1: Apply the fundamentals of parallel and distributed computing including parallel architectures and paradigms.

CO2: Apply parallel algorithms and key technologies.

CO3: Develop and execute basic parallel and distributed applications using basic programming models and tools.

CO4: Analyze the performance issues in parallel computing and trade-offs.

Text Books:

1. C Lin, L Snyder. Principles of Parallel Programming. USA: Addison-Wesley (2008).
2. A Grama, A Gupta, G Karypis, V Kumar. Introduction to Parallel Computing, Addison Wesley (2003).
3. B Gaster, L Howes, D Kaeli, P Mistry, and D Schaa. Heterogeneous Computing with OpenCL. Morgan Kaufmann and Elsevier (2011).
4. T Mattson, B Sanders, B Massingill. Patterns for Parallel Programming. Addison-Wesley (2004).
5. Quinn, M. J., Parallel Programming in C with MPI and OpenMP, McGraw-Hill (2004)

Natural Language Processing (ECSE120)

Credit: 3+0

Course Objective: Course Objectives: To understand the advanced concepts of Natural Language Processing and to be able to apply the various concepts of NLP in other application areas.

Syllabus

- 1. Introduction:** Origins and challenges of NLP – Language Modeling: Grammar-based LM, Statistical LM – Regular Expressions, Finite-State Automata – English Morphology, Transducers for lexicon and rules, Tokenization, Detecting and Correcting Spelling Errors, Minimum Edit Distance, WORD LEVEL ANALYSIS : Unsmoothed N-grams, Evaluating N-grams, Smoothing, Interpolation and Backoff – Word Classes, Part-of-Speech Tagging, Rule-based, Stochastic and Transformation-based tagging, Issues in PoS tagging – Hidden Markov and Maximum Entropy models.
- 2. Syntactic Analysis:** Context Free Grammars, Grammar rules for English, Treebanks, Normal Forms for grammar – Dependency Grammar – Syntactic Parsing, Ambiguity, Dynamic Programming parsing – Shallow parsing – Probabilistic CFG, Probabilistic CYK, Probabilistic Lexicalized CFGs – Feature structures, Unification of feature structures.
- 3. Semantics and Pragmatics:** Requirements for representation, First-Order Logic, Description Logics – Syntax-Driven Semantic analysis, Semantic attachments – Word Senses, Relations between Senses, Thematic Roles, selectional restrictions – Word Sense Disambiguation, WSD using Supervised, Dictionary & Thesaurus, Bootstrapping methods – Word Similarity using Thesaurus and Distributional methods.
- 4. Basic Concepts of Speech Processing:** Speech Fundamentals: Articulatory Phonetics – Production and Classification of Speech Sounds; Acoustic Phonetics – Acoustics of Speech Production; Review of Digital Signal Processing Concepts; Short-Time Fourier Transform, Filter-Bank and LPC Methods. **Speech-Analysis:** Features, Feature Extraction and Pattern Comparison Techniques: Speech Distortion Measures– Mathematical and Perceptual – Log-Spectral Distance, Cepstral Distances, Weighted Cepstral Distances and Filtering, Likelihood Distortions, Spectral Distortion Using a Warped Frequency Scale, LPC, PLP And MFCC Coefficients, Time Alignment and Normalization – Dynamic Time Warping, Multiple Time – Alignment Paths. **SPEECH MODELING:** Hidden Markov Models: Markov Processes, HMMs – Evaluation, Optimal State Sequence – Viterbi Search, Baum-Welch Parameter Re-Estimation, Implementation Issues.

Course Outcome (CO): At the end of course, the student will be able

CO1: To learn the fundamentals of natural language processing.

CO2: To understand the use of CFG and PCFG in NLP.

CO3: To understand the role of semantics of sentences and pragmatic.

CO4: To Introduce Speech Production and Related Parameters of Speech.

CO5: To show the computation and use of techniques such as short time Fourier transform, linear predictive coefficients and other coefficients in the analysis of speech.

Text Books:

1. Daniel Jurafsky, James H. Martin—Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics and Speech, Pearson Publication, 2014.
2. Steven Bird, Ewan Klein and Edward Loper, —Natural Language Processing with Python, First Edition, O'Reilly Media, 2009.
3. Lawrence Rabiner and Biing-Hwang Juang, “Fundamentals of Speech Recognition”, Pearson Education, 2003.

4. Daniel Jurafsky and James H Martin, "Speech and Language Processing – An Introduction To Natural Language Processing, Computational Linguistics, And Speech Recognition", Pearson Education, 2002.
5. Frederick Jelinek, "Statistical Methods of Speech Recognition", MIT Press, 1997.
6. Breck Baldwin, –Language Processing with Java and Ling Pipe Cookbook, Atlantic Publisher, 2015
7. Richard M Reese, –Natural Language Processing with Java, OReilly Media, 2015.
8. Nitin Indurkha and Fred J. Damerau, –Handbook of Natural Language Processing, Second Edition, Chapman and Hall/CRC Press, 2010.
9. Tanveer Siddiqui, U.S. Tiwary, –Natural Language Processing and Information Retrieval, Oxford University Press, 2008.

High Performance Computing (ECSE121)

Credit: 3+0

Course Objective: The objective of this course is to teach participants the basic concepts of parallel computing and equip them with knowledge which will be sufficient enough for them to write parallel programs using industry-standard parallel programming frameworks.

Syllabus

- 1. Overview of Grid Computing Technology & Open Grid Services Architecture:** History of Grid Computing, High Performance Computing, Cluster Computing. Peer-to-Peer Computing, Internet Computing, Grid Computing Model and Protocols, Types of Grids: Desktop Grids, Cluster Grids, Data Grids, High- Performance Grids, Applications and Architectures of High-Performance Grids, High Performance Application Development Environment. **Open Grid Services Architecture:** Introduction, Requirements, Capabilities, Security Considerations, GLOBUS Toolkit
- 2. Overview of Cluster Computing:** Cluster Computer and its Architecture, Clusters Classifications, Components for Clusters, Cluster Middleware and SSI, Resource Management and Scheduling, Programming, Environments and Tools, Cluster Applications, Cluster Systems,
- 3. Beowulf Cluster:** The Beowulf Model, Application Domains, Beowulf System Architecture, Software Practices, Parallel Programming with MPL, Parallel Virtual Machine (PVM).
- 4. Overview of Cloud Computing:** Types of Cloud, Cyber infrastructure, Service Oriented Architecture Cloud Computing Components: Infrastructure, Storage, Platform, Application, Services, Clients, Cloud Computing Architecture.

Course Outcome (CO) : At the end of course, the student will be able

CO1: To understand the basic concept of Computer architecture and Modern Processor.

CO2: To understand the basic concepts of access optimization and parallel computers.

CO3: To describe different parallel processing platforms involved in achieving high performance computing.

CO4: Develop efficient and high-performance parallel programming.

CO5: Able to learn parallel programming using message passing paradigm.

Text Books:

1. Laurence T. Yang, Minyi Guo – High Performance Computing Paradigm and Infrastructure John Wiley
2. Ahmar Abbas, “Grid Computing: Practical Guide to Technology & Applications”, Firewall Media, 2004.
3. Joshy Joseph and Craig Fellenstein , “Grid Computing” Pearson Education, 2004.
4. Ian Foster, et al., “The Open Grid Services Architecture”, Version 1.5 (GFD.80). Open Grid Forum, 2006.
5. Rajkumar Buyya. High Performance Cluster Computing: Architectures and Systems. Prentice Hall India, 1999.

Design & Development of Mobile Applications (ECSE122)

Credit: 3+0

Course Objective: A course in Design & Development of Applications with an emphasis on software development. Integrated development tools, software development kits, and software subsystems are to be employed to develop database, eCommerce, and mobile applications.

Syllabus

- 1. Introduction:** Introduction to mobile applications – Embedded systems - Market and business drivers for mobile applications – Publishing and delivery of mobile applications – Requirements gathering and validation for mobile applications.
- 2. Basic Design:** Introduction – Basics of embedded systems design – Embedded OS – Design constraints for mobile applications, both hardware and software related – Architecting mobile applications – User interfaces for mobile applications – touch events and gestures – Achieving quality constraints – performance, usability, security, availability and modifiability.
- 3. Advanced Design:** Designing applications with multimedia and web access capabilities – Integration with GPS and social media networking applications – Accessing applications hosted in a cloud computing environment – Design patterns for mobile applications.
- 4. Introduction to Android And Objective C:** Establishing the development environment – Android architecture – Activities and views – Interacting with UI – Persisting data using SQLite – Packaging and deployment – Interaction with server-side applications – Using Google Maps, GPS and Wi-Fi – Integration with social media applications. **INTRODUCTION TO OBJECTIVE C:** iOS features, UI implementation, Touch frameworks – Data persistence using Core Data and SQLite – Location aware applications using Core Location and Map Kit – Integrating calendar and address book with social media application – Using Wi-Fi - iPhone marketplace. Swift: Introduction to Swift, features of swift.

Course Outcome (CO): At the end of course, the student will be

CO1: Able to expose the technology and business trends impacting mobile applications.

CO2: Competent with the characterization and architecture of mobile applications.

CO3: Competent with understanding enterprise scale requirements of mobile applications.

CO4: Competent with designing and developing mobile applications using one application development framework.

CO5: Able to expose the Android and iOS platforms to develop the mobile applications

Text Books:

1. Charlie Collins, Michael Galpin and Matthias Kappler, "Android in Practice", DreamTech, 2012
2. AnubhavPradhan , Anil V Despande Composing Mobile Apps,Learn ,explore,apply
3. James Dovey and Ash Furrow, "Beginning Objective C", Apress, 2012
4. Jeff McWherter and Scott Gowell, "Professional Mobile Application Development", Wrox, 2012
5. David Mark, Jack Nutting, Jeff LaMarche and Frederic Olsson, "Beginning iOS
6. Development: Exploring the iOS SDK", Apress, 2013.

Introduction to Software Testing (ECSE123)

Credit: 3+0

Course Objective:

- To study fundamental concepts in software testing
- To discuss various software testing issues and solutions in software unit test, integration and system testing.
- To expose the advanced software testing topics, such as object-oriented software testing methods.

Syllabus

- 1. Review of Software Engineering:** Overview of Software Evolution, SDLC, Testing Process, Terminologies in Testing: Error, Fault, Failure, Verification, Validation, Difference Between Verification and Validation, Test Cases, Testing Suite, Test, Oracles, Impracticality of Testing All Data; Impracticality of Testing All Paths. Verification: Verification Methods, SRS Verification, Source Code Reviews, User Documentation Verification, Software, Project Audit, Tailoring Software Quality Assurance Program by Reviews, Walkthrough, Inspection and Configuration Audits.
- 2. Functional Testing:** Boundary Value Analysis, Equivalence Class Testing, Decision Table Based Testing, Cause Effect Graphing Technique. Structural Testing: Control Flow Testing, Path Testing, Independent Paths, Generation of Graph from Program, Identification of Independent Paths, Cyclomatic Complexity, Data Flow Testing, Mutation Testing.
- 3. Regression Testing:** What is Regression Testing? Regression Test cases selection, Reducing the number of test cases, Code coverage prioritization technique. Reducing the number of test cases: Prioritization guidelines, Priority category, Scheme, Risk Analysis.
- 4. Software Testing Activities & Object-Oriented Testing:** Levels of Testing, Debugging, Testing techniques and their applicability, Exploratory Testing Automated Test Data Generation: Test Data, Approaches to test data generation, test data generation using genetic algorithm, Test Data Generation Tools, Software Testing Tools, and Software test Plan. **Object Oriented Testing:** Definition, Issues, Class Testing, Object Oriented Integration and System Testing. Testing Web Applications: Web Testing, User Interface Testing, Usability Testing, Security Testing, Performance Testing, Database testing, Post Deployment Testing.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Have an ability to apply software testing knowledge and engineering methods.

CO2: Have an ability to design and conduct a software test process for a software testing project.

CO3: Have an ability to identify the needs of software test automation, and define and develop a test tool to support test automation.

CO4: Have an ability understand and identify various software testing problems, and solve these Problems by designing and selecting software test models, criteria, strategies, and methods.

CO5: Have basic understanding and knowledge of contemporary issues in software testing, such as Component-based software testing problems.

Text Books:

1. Yogesh Singh, "Software Testing", Cambridge University Press, New York, 2012
2. K.K. Aggarwal & Yogesh Singh, "Software Engineering", New Age International Publishers, New Delhi, 2003.
3. Roger S. Pressman, "Software Engineering – A Practitioner's Approach", Fifth Edition, McGraw-Hill International Edition, New Delhi, 2001.
4. Marc Roper, "Software Testing", McGraw-Hill Book Co., London, 1994.
5. M.C. Trivedi, Software Testing & Audit, Khanna Publishing House
6. Boris Beizer, "Software System

Testing and Quality Assurance”, Van Nostrand Reinhold, New York, 1984

Distributed System (ECSE124)

Credit: 3+0

Course Objective: This course is an introduction to the design of distributed systems and algorithms that support distributed computing.

Syllabus

- 1. Characterization of Distributed Systems:** Introduction, Examples of distributed Systems, Resource sharing and the Web Challenges. Architectural models, Fundamental Models. Theoretical Foundation for Distributed System: Limitation of Distributed system, absence of global clock, shared memory, Logical clocks, Lamport's & vectors logical clocks. Concepts in Message Passing Systems: causal order, total order, total causal order, Techniques for Message Ordering, Causal ordering of messages, global state, and termination detection.
- 2. Distributed Mutual Exclusion:** Classification of distributed mutual exclusion, requirement of mutual exclusion theorem, Token based and non-token-based algorithms, performance metric for distributed mutual exclusion algorithms. Distributed Deadlock Detection: system model, resource Vs communication deadlocks, deadlock prevention, avoidance, detection & resolution, centralized dead lock detection, distributed dead lock detection, path pushing algorithms, edge chasing algorithms.
- 3. Agreement Protocols:** Introduction, System models, classification of Agreement Problem, Byzantine agreement problem, Consensus problem, Interactive consistency Problem, Solution to Byzantine Agreement problem, Application of Agreement problem, Atomic Commit in Distributed Database system. Distributed Resource Management: Issues in distributed File Systems, Mechanism for building distributed file systems, Design issues in Distributed Shared Memory, Algorithm for Implementation of Distributed Shared Memory.
- 4. Failure Recovery in Distributed Systems & Transactions and Concurrency Control:** Concepts in Backward and Forward recovery, Recovery in Concurrent systems, obtaining consistent Checkpoints, Recovery in Distributed Database Systems. Fault Tolerance: Issues in Fault Tolerance, Commit Protocols, Voting protocols, Dynamic voting protocols. **Transactions and Concurrency Control:** Transactions, Nested transactions, Locks, Optimistic Concurrency control, Timestamp ordering, Comparison of methods for concurrency control. Distributed Transactions: Flat and nested distributed transactions, Atomic Commit protocols, Concurrency control in distributed transactions, Distributed deadlocks, Transaction recovery. Replication: System model and group communication, Fault - tolerant services, highly available services, Transactions with replicated data.

Course Outcome (CO) : At the end of course, the student will be able to

CO1: Provide hardware and software issues in modern distributed systems.

CO2: Get knowledge in distributed architecture, naming, synchronization, consistency and replication, fault tolerance, security, and distributed file systems.

CO3: Analyze the current popular distributed systems such as peer-to-peer (P2P) systems will also be analyzed.

CO4: Know about Shared Memory Techniques and have sufficient knowledge about file access.

CO5: Have knowledge of Synchronization and Deadlock.

Text Books:

1. Singhal & Shivaratri, "Advanced Concept in Operating Systems", McGraw Hill
2. Ramakrishna, Gehrke, "Database Management Systems", McGraw Hill
3. Vijay K. Garg Elements of Distributed Computing, Wiley
4. Coulouris, Dollimore, Kindberg, "Distributed System: Concepts and Design", Pearson Education
5. Tenenbaum, Steen, "Distributed Systems", PHI

Mobile Computing (ECSE125)

Credit: 3+0

Course Objective:

- To make the student understand the concept of mobile computing paradigm, its novel applications and limitations.
- To understand the typical mobile networking infrastructure through a popular GSM protocol.
- To understand the issues and solutions of various layers of mobile networks, namely MAC layer, Network Layer & Transport Layer.
- To understand the database issues in mobile environments & data delivery models.
- To understand the ad hoc networks and related concepts.
- To understand the platforms and protocols used in mobile environment.

Syllabus

1. **Introduction:** Issues in mobile computing, overview of wireless telephony: cellular concept, GSM: air-interface, channel structure, location management: HLR-VLR, hierarchical, handoffs, channel allocation in cellular systems, CDMA, GPRS.
2. **Wireless Networking, Wireless LAN Overview:** MAC issues, IEEE 802.11, Blue Tooth, Wireless multiple access protocols, TCP over wireless, Wireless applications, data broadcasting, Mobile IP, WAP: Architecture, protocol stack, application environment, applications.
3. **Data Management Issues & Mobile Agents:** Data replication for mobile computers, adaptive clustering for mobile wireless networks, File system, Disconnected operations. **Mobile Agents:** computing, security and fault tolerance, transaction processing in mobile computing environment.
4. **Ad Hoc Networks:** Localization, MAC issues, Routing protocols, global state routing (GSR), Destination sequenced distance vector routing (DSDV), Dynamic source routing (DSR), Ad Hoc on demand distance vector routing (AODV), Temporary ordered routing algorithm (TORA), QoS in Ad Hoc Networks, applications.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Explain and discuss issues in mobile computing and illustrate overview of wireless telephony and channel allocation in cellular systems.

CO2: Explore the concept of Wireless Networking and Wireless LAN.

CO3: Analyze and comprehend Data management issues like data replication for mobile computers, adaptive clustering for mobile wireless networks and Disconnected operations.

CO4: Identify Mobile computing Agents and state the issues pertaining to security and fault tolerance in mobile computing environment.

CO5: Compare and contrast various routing protocols and will identify and interpret the performance of network systems using Adhoc networks.

Text books:

1. J. Schiller, Mobile Communications, Addison Wesley.
2. A. Mehrotra, GSM System Engineering.
3. M. V. D. Heijden, M. Taylor, Understanding WAP, Artech House.
4. Charles Perkins, Mobile IP, Addison Wesley.
5. Charles Perkins, Ad hoc Networks, Addison Wesley.

Wireless Network Systems (ECSE126)

Credit: 3+0

Course Objective:

- To study the evolving wireless technologies and standards
- To understand the architectures of various access technologies such as 3G, 4G.
- To understand various protocols and services provided by next generation networks.

Syllabus

- 1. Introduction to WLAN Technologies (IEEE802.11):** System architecture, protocol architecture, 802.11b, 802.11a – Hiper LAN: WATM, BRAN, HiperLAN2 – Bluetooth: Architecture, WPAN – IEEE 802.15.4, Wireless USB, Zigbee, 6LoWPAN, Wireless HART.**MOBILE NETWORK LAYER** Introduction to Mobile IP, IP packet delivery, Agent discovery, tunneling and encapsulation, IPV6- Network layer in the internet- Mobile IP session initiation protocol - mobile ad-hoc network: Routing: Destination Sequence distance vector, IoT: CoAP
- 2. 3G Overview** of UTM Terrestrial Radio access network-UMTS Core network Architecture: 3GPP Architecture, User equipment, CDMA2000 overview- Radio and Network components, Network structure, Radio Network, TD-CDMA, TD – SCDMA.
- 3. Internetworking between WLANS and WWANS** Internetworking objectives and requirements, Schemes to connect WLANS and 3G Networks, Session Mobility, Internetworking Architecture for WLAN and GPRS, System Description, Local Multipoint Distribution Service, Multichannel Multipoint Distribution System.
- 4. 4G & Beyond** Introduction – 4G vision – 4G features and challenges - Applications of 4G – 4G Technologies: Multicarrier Modulation, Smart antenna techniques, IMS Architecture, LTE, Advanced Broadband Wireless Access and Services, MVNO.

Course Outcome (CO) : Upon completion of the course, the student would be able to

CO1: Conversant with the latest 3G/4G networks and its architecture

CO2: Design and implement wireless network environment for any application using latest wireless protocols and standards

CO3: Ability to select the suitable network depending on the availability and requirement

CO4: Implement different type of applications for smart phones and mobile devices with latest network strategies

Text Books:

1. Jochen Schiller, Mobile Communications, Second Edition, Pearson Education 2012.
2. Vijay Garg, –Wireless Communications and networking, First Edition, Elsevier 2007.
3. Erik Dahlman, Stefan Parkvall, Johan Skold and Per Beming, "3G Evolution HSPA and LTE for Mobile Broadband, Second Edition, Academic Press, 2008.
4. Anurag Kumar, D.Manjunath, Joy kuri, –Wireless Networking, First Edition, Elsevier 2011.
5. Simon Haykin , Michael Moher, David Koilpillai, –Modern Wireless Communications, First Edition, Pearson Education 2013

Elective-2

(Offered by Department)

Randomized Algorithms (ECSE214)

Credit: 3+0

Course Objective: Randomized Algorithms are the state of the art in contemporary algorithm design. They are usually simple, sometimes even easy to analyze, and they work well in practice. This course provides an introduction to basic concepts in the design and analysis of randomized algorithms.

Syllabus

1. **Randomized Data Structures:** Random Treaps, Skip Lists, Hash Tables, Universal Family of Hash Functions, Perfect Hashing.
2. **Randomized Computational Geometry:** Illustrations of randomized incremental algorithms like randomized convex hull construction, geometric duality, half space intersections, Delaunay Triangulation, trapezoidal decomposition; illustrations of random sampling like point location in arrangements, and linear programming.
3. **Online Algorithms & Distributed Algorithms:** Adversary models, online paging against oblivious and adaptive adversaries, Yao's minimax principle, lower bound for online paging against an oblivious adversary, the k-server problem. **Distributed Algorithms:** Symmetry breaking problems like leader election, Byzantine agreement, maximal independent set, and colouring; algorithms for dynamic networks; the k-machine model for processing large graphs.
4. **Streaming Algorithms:** the streaming model, approximate counting, reservoir sampling, AMS sketching. **Property testing algorithms:** the property testing model, testing whether a graph is connected, bipartite (enforce and test paradigm), and triangle free (using Szemerédi's regularity lemma).

Course Outcomes (CO): At the end of this course, students will be able to

CO1: Design and analyze algorithms for basic numerical problems.

CO2: Use randomization in the design and analysis of efficient algorithms.

CO3: Efficiently solve optimization problems using approximation algorithms.

CO4: Identify problems that are unlikely to admit efficient algorithms and argue for their difficulty via computational complexity theory.

CO5: Identify parameters in computational problems that enable efficient algorithms, design and analyze parameterized algorithms.

Text Books:

1. Randomized Algorithms, by Motwani and Raghavan, Cambridge University Press, 1995.
2. Probability and Computing: Randomized Algorithms and Probabilistic Analysis, by Mitzenmacher and Upfal, Cambridge University Press, 2nd edition, 2017.

Programming for Embedded Systems (ECSE215)

Credit: 3+0

Course Objective: This course focuses on the programming aspects for embedded system.

Syllabus

1. **Embedded 'C' Programming:** Bitwise operations, Dynamic memory allocation, OS services Linked stack and queue, Sparse matrices, Binary tree, Interrupt handling in C, Code optimization issues, - Writing LCD drives, LED drivers, Drivers for serial port communication, - Embedded Software Development Cycle and Methods (Waterfall, Agile). **C++ Programming:** 'cin', 'cout', formatting and I/O manipulators, new and delete operators, Defining a class, data members and methods, 'this' pointer, constructors, destructors, friend function, dynamic memory allocation.
2. **Overloading and Inheritance:** Need of operator overloading, overloading the assignment, overloading using friends, type conversions, single inheritance, base and derived classes, friend classes, types of inheritance, hybrid inheritance, multiple inheritance, virtual base class, polymorphism, virtual functions.
3. **Templates:** Function template and class template, member function templates and template arguments, Exception Handling: syntax for exception handling code: try-catch-throw, Multiple Exceptions.
4. **Scripting Languages:** Overview of Scripting Languages – PERL, CGI, VB Script, Java Script. PERL: Operators, Statements Pattern Matching etc. Data Structures, Modules, Objects, Tied Variables, Inter process Communication Threads, Compilation & Line Interfacing.

Course Outcomes (CO): At the end of this course, students will be able to

CO1: Write an embedded C application of moderate complexity.

CO2: Develop and analyze algorithms in C++.

CO3: Differentiate interpreted languages from compiled languages.

Text Books:

1. Michael J. Pont, "Embedded C", Pearson Education, 2nd Edition, 2008
2. Randal L. Schwartz, "Learning Perl", O'Reilly Publications, 6th Edition 2011
3. A. Michael Berman, "Data structures via C++", Oxford University Press, 2002
4. Robert Sedgewick, "Algorithms in C++", Addison Wesley Publishing Company, 1999
5. Abraham Silberschatz, Peter B, Greg Gagne, "Operating System Concepts", John Wiley & Sons, 2005

Introduction to Deep Learning (ECSE216)

Credit: 3+0

Course Objective: Deep learning is a class of machine learning algorithms which enables computers to learn from examples. Deep learning techniques have been used successfully for variety of applications, including: automatic speech recognition, image recognition, natural language processing, drug discovery, and recommendation systems. In this course, students will learn the fundamentals of deep learning, and the main research activities in this field.

Syllabus

- 1. Introduction to Deep Networks:** Introduction to machine learning- Linear models (SVMs and Perceptrons, logistic regression)- Intro to Neural Nets: What a shallow network computes- Training a network: loss functions, back propagation and stochastic gradient descent- Neural networks as universal function approximates. **DEEP NETWORKS:** History of Deep Learning- A Probabilistic Theory of Deep Learning- Backpropagation and regularization, batch normalization- VC Dimension and Neural Nets-Deep Vs Shallow Networks-Convolutional Networks- Generative Adversarial Networks (GAN), Semi-supervised Learning.
- 2. Dimensionality Reduction:** Linear (PCA, LDA) and manifolds, metric learning – Auto encoders and dimensionality reduction in networks - Introduction to Convnet - Architectures – AlexNet, VGG, Inception, ResNet - Training a Convnet: weights initialization, batch normalization, hyper parameter optimization.
- 3. Optimization and Generalization:** Optimization in deep learning- non-convex optimization for deep networks- Stochastic Optimization Generalization in neural networks- Spatial Transformer Networks- Recurrent networks, LSTM - Recurrent Neural Network Language Models- Word-Level RNNs & Deep Reinforcement Learning - Computational & Artificial Neuroscience.
- 4. Case Study and Applications:** Image net- Detection-Audio Wave Net-Natural Language Processing Word2Vec - Joint Detection-Bioinformatics- Face Recognition- Scene Understanding- Gathering Image Captions.

Course Outcome (CO): At the end of course, the student will be able

CO1: To present the mathematical, statistical and computational challenges of building neural networks.

CO2: To study the concepts of deep learning.

CO3: To introduce dimensionality reduction techniques.

CO4: To enable the students to know deep learning techniques to support real-time applications.

CO5: To examine the case studies of deep learning techniques.

Text Books:

1. Cosma Rohilla Shalizi, Advanced Data Analysis from an Elementary Point of View, 2015.
2. Deng & Yu, Deep Learning: Methods and Applications, Now Publishers, 2013.
3. Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, MIT Press, 2016.
4. Michael Nielsen, Neural Networks and Deep Learning, Determination Press, 2015.

Service Oriented Architecture (ECSE217)

Credit: 3+0

Course Objective: To introduce the concepts and design principles of SOA, Non-technical aspects such as governance, impact on culture and organization, as well as the various interoperability standards, technology infrastructure and security considerations associated with SOA implementations.

Syllabus

- 1. Introduction:** SOA and MSA Basics: Service Orientation in Daily Life, Evolution of SOA and MSA. Service oriented Architecture and Microservices architecture – Drivers for SOA, Dimensions of SOA, Conceptual Model of SOA, Standards and Guidelines for SOA, Emergence of MSA. Enterprise-Wide SOA: Considerations for Enterprise-wide SOA, Strawman Architecture for Enterprise-wide SOA, Enterprise SOA Reference Architecture, Object-oriented Analysis and Design (OOAD) Process, Service-oriented Analysis and Design (SOAD) Process, SOA Methodology for Enterprise.
- 2. Service-Oriented Applications:** Considerations for Service-oriented Applications, Patterns for SOA, Pattern-based Architecture for Service-oriented Applications, Composite Applications, Composite Application Programming Model. Service-Oriented Analysis and Design: Need for Models, Principles of Service Design, Nonfunctional Properties for Services, Design of Activity Services (or Business Services), Design of Data Services, Design of Client Services, Design of Business Process Services.
- 3. Technologies for SOA:** Technologies for Service Enablement, Technologies for Service Integration, Technologies for Service Orchestration. SOA Governance and Implementation: Strategic Architecture Governance, Service Design-time Governance, Service Run-time Governance, Approach for Enterprise-wide SOA Implementation.
- 4. Big Data and SOA with Best Practices:** Concepts, Big Data and its characteristics, Technologies for Big Data, Service-orientation for Big Data Solutions. Business Case for SOA: Stakeholder Objectives, Benefits of SOA, Cost Savings, Return on Investment (ROI), Build a Case for SOA. **SOA Best Practices:** SOA Strategy – Best Practices, SOA Development – Best Practices, SOA Governance – Best Practices. EA and SOA for Business and IT Alignment: Enterprise Architecture, Need for Business and It Alignment, EA and SOA for Business and It Alignment.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Comprehend the need for SOA and its systematic evolution.

CO2: Apply SOA technologies to enterprise domain.

CO3: Design and analyze various SOA patterns and techniques.

CO4: Compare and evaluate best strategies and practices of SOA.

CO5: Understand the business case for SOA.

Text Books:

1. Shankar Kambhampaty; Service - Oriented Architecture & Microservices Architecture: For Enterprise, Cloud, Big Data and Mobile; Wiley; 3rd Edition; 2018; ISBN: 9788126564064.
2. Icon Group International; The 2018-2023 World Outlook for Service-Oriented Architecture (SOA) Software and Services; ICON Group International; 1st Edition, 2017; ASIN: B06WGPN8YD.
3. Thomas Erl; Service Oriented Architecture Concepts Technology & Design; Pearson Education Limited; 2015; ISBN-13: 9788131714904.
4. Guido Schmutz, Peter Welkenbach, Daniel Liebhart; Service Oriented Architecture: An Integration Blueprint; Shroff Publishers & Distributors; 2010; ISBN-13: 9789350231081.

Quantum Computing (ECSE218)

Credit: 3+0

Course Objective: This course focuses on the fundamentals of Quantum Computing and its application in information technology.

Syllabus

- 1. Fundamental Concepts & Quantum Computation:** Global Perspectives, Quantum Bits, Quantum Computation, Quantum Algorithms, Quantum Information, Postulates of Quantum Mechanisms. **Quantum Computation:** Quantum Circuits – Quantum algorithms, Single Orbit operations, Control Operations, Measurement, Universal Quantum Gates, Simulation of Quantum Systems, Quantum Fourier transform, Phase estimation, Applications, Quantum search algorithms – Quantum counting – Speeding up the solution of NP – complete problems – Quantum Search for an unstructured database.
- 2. Quantum Computers:** Guiding Principles, Conditions for Quantum Computation, Harmonic Oscillator Quantum Computer, Optical Photon Quantum Computer – Optical cavity Quantum electrodynamics, Ion traps, Nuclear Magnetic resonance.
- 3. Quantum Information:** Quantum noise and Quantum Operations – Classical Noise and Markov Processes, Quantum Operations, Examples of Quantum noise and Quantum Operations – Applications of Quantum operations, Limitations of the Quantum operations formalism, Distance Measures for Quantum information.
- 4. Quantum Error Correction:** Introduction, Shor code, Theory of Quantum Error –Correction, Constructing Quantum Codes, Stabilizer codes, Fault – Tolerant Quantum Computation, Entropy and information – Shannon Entropy, Basic properties of Entropy, Von Neumann, Strong Sub Additivity, Data Compression, Entanglement as a physical resource.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Distinguish problems of different computational complexity and explain why certain problems are rendered tractable by quantum computation with reference to the relevant concepts in quantum theory.

CO2: Demonstrate an understanding of a quantum computing algorithm by simulating it on a classical computer, and state some of the practical challenges in building a quantum computer.

CO3: Contribute to a medium-scale application program as part of a co-operative team, making use of appropriate collaborative development tools (such as version control systems).

CO4: Produce code and documentation that is comprehensible to a group of different programmers and present the theoretical background and results of a project in written and verbal form.

CO5: Apply knowledge, skills, and understanding in executing a defined project of research, development, or investigation and in identifying and implementing relevant outcomes.

Text Books:

1. Micheal A. Nielsen. & Issac L. Chiang, "Quantum Computation and Quantum Information", Cambridge University Press, Fint South Asian edition, 2002.
2. Eleanor G. Rieffel, Wolfgang H. Polak, "Quantum Computing - A Gentle Introduction" (Scientific and Engineering Computation) Paperback – Import, Oct 2014
3. Computing since Democritus by Scott Aaronson, Computer Science: An Introduction by N. David Mermin
4. Yanofsky's and Mannucci, Quantum Computing for Computer Scientists.

Introduction to Cloud Computing (ECSE219)

Credit: 3+0

Course Objective: This course gives students an insight into the basics of cloud computing along with virtualization, cloud computing is one of the fastest growing domains from a while now. It will provide the students basic understanding about cloud and virtualization along with it how one can migrate over it.

Syllabus

- 1. Introduction to Cloud Computing:** Definition of Cloud – Evolution of Cloud Computing – Underlying Principles of Parallel and Distributed Computing – Cloud Characteristics – Elasticity in Cloud – On-demand Provisioning.
- 2. Cloud Enabling Technologies Service Oriented Architecture:** REST and Systems of Systems – Web Services – Publish, Subscribe Model – Basics of Virtualization – Types of Virtualizations – Implementation Levels of Virtualization – Virtualization Structures – Tools and Mechanisms – Virtualization of CPU – Memory – I/O Devices – Virtualization Support and Disaster Recovery.
- 3. Cloud Architecture, Services and Storage:** Layered Cloud Architecture Design – NIST Cloud Computing Reference Architecture – Public, Private and Hybrid Clouds – IaaS – PaaS – SaaS – Architectural Design Challenges – Cloud Storage – Storage-as-a-Service – Advantages of Cloud Storage – Cloud Storage Providers – S3.
- 4. Resource Management and Security in Cloud & Cloud Technologies and Advancements Hadoop:** Inter Cloud Resource Management – Resource Provisioning and Resource Provisioning Methods – Global Exchange of Cloud Resources – Security Overview – Cloud Security Challenges – Software-as-a-Service Security – Security Governance – Virtual Machine Security – IAM – Security Standards. **Cloud Technologies and Advancements Hadoop:** MapReduce – Virtual Box – Google App Engine – Programming Environment for Google App Engine – Open Stack – Federation in the Cloud – Four Levels of Federation – Federated Services and Applications – Future of Federation.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Describe architecture and underlying principles of cloud computing.

CO2: Explain need, types and tools of Virtualization for cloud.

CO3: Describe Services Oriented Architecture and various types of cloud services.

CO4: Explain Inter cloud resources management cloud storage services and their providers
Assess security services and standards for cloud computing.

CO5: Analyze advanced cloud technologies.

Text Books:

1. Kai Hwang, Geoffrey C. Fox, Jack G. Dongarra, “Distributed and Cloud Computing, From Parallel Processing to the Internet of Things”, Morgan Kaufmann Publishers, 2012.
2. Rittinghouse, John W., and James F. Ransome, —Cloud Computing: Implementation, Management and Security, CRC Press, 2017.
3. Rajkumar Buyya, Christian Vecchiola, S. ThamaraiSelvi, —Mastering Cloud Computing, Tata Mcgraw Hill, 2013.
4. Toby Velte, Anthony Velte, Robert Elsenpeter, “Cloud Computing – A Practical Approach, Tata Mcgraw Hill, 2009.
5. George Reese, “Cloud Application Architectures: Building Applications and Infrastructure in the Cloud: Transactional Systems for EC2 and Beyond (Theory in Practice), O’Reilly, 2009.

Blockchain Architecture Design (ECSE220)

Credit: 3+0

Course Objective: This course is an introduction to Blockchain Architecture and its application in finance and governance.

Syllabus

- 1. Introduction to Blockchain & Consensus:** Digital Money to Distributed Ledgers, Design Primitives: Protocols, Security, Consensus, Permissions, Privacy. Blockchain Architecture and Design: Basic crypto primitives: Hash, Signature,) Hash chain to Blockchain, Basic consensus mechanisms. **Consensus:** Requirements for the consensus protocols, Proof of Work (PoW), Scalability aspects of Blockchain consensus protocols Permissioned Block chains: Design goals, Consensus protocols for Permissioned Blockchains.
- 2. Hyperledger Fabric (A):** Decomposing the consensus process, Hyperledger fabric components, Chain code Design and Implementation Hyperledger Fabric (B): Beyond Chain code: fabric SDK and Front End (b) Hyperledger composer tool.
- 3. Blockchain in Financial Software and Systems (FSS):** (i) Settlements, (ii) KYC, (iii) Capital markets, (iv) Insurance. **Blockchain in Trade/Supply Chain:** (i) Provenance of goods, visibility, trade/supply chain finance, invoice management discounting, etc.
- 4. Blockchain for Government:** (i) Digital identity, land records and other kinds of record keeping between government entities, (ii) public distribution system social welfare systems Blockchain Cryptography, Privacy and Security on Blockchain.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Describe the basic understanding of Blockchain architecture along with its primitive.

CO2: Explain the requirements for basic protocol along with scalability aspects.

CO3: Design and deploy the consensus process using frontend and backend.

CO4: Apply Blockchain techniques for different use cases like Finance, Trade/Supply and Government activities.

Text Books:

1. Mastering Bitcoin: Unlocking Digital Cryptocurrencies, by Andreas Antonopoulos
2. Blockchain by Melanie Swa, O'Reilly
3. Hyperledger Fabric - <https://www.hyperledger.org/projects/fabric>
4. Zero to Blockchain - An IBM Redbooks course, by Bob Dill, David Smits - <https://www.redbooks.ibm.com/Redbooks.nsf/RedbookAbstracts/crse0401.html>

Distributed Algorithms (ECSE221)

Credit: 3+0

Course Objective: The course provides students with the foundation knowledge to understand, analysis and design distributed algorithms. This shall be useful to a wide variety of research topics from the theory of distributed algorithms to protocol design, e.g., broadcasting protocols for discovery purposes in ad-hoc networks.

Syllabus

1. **Review of Prerequisite Topics:** Graph theory, probability theory covering Markov's inequality, Chebyshev's inequality, Chernoff bounds, Markov chains and random walks.
2. **Models for Distributed Computer Networks:** Message passing and shared memory models, synchronous and asynchronous timing models, failure models. Complexity measures like time, space, and message complexity.
3. **Fundamental Problems on Distributed Networks:** Maximal independent set, minimum spanning tree, vertex colouring, dominating set, routing algorithms, leader election, Byzantine agreement, synchronizers, graph spanners, dynamic networks.
4. **Application Specific Problems:** Storage and retrieval of data in peer-to-peer computing, coverage and routing in sensor networks, and rumour spreading in social networking.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Demonstrate basic concepts and principles Distributed Algorithm.

CO2: Apply Distributed Algorithm in Computer Networks.

CO3: Demonstrate challenges in Distributed Algorithm.

CO4: Apply Distributed Algorithms in storage and retrieval of information.

Text Books:

1. Distributed Computing: a Locality-Sensitive Approach, by David Peleg.
2. Distributed Algorithms, by Nancy Lynch.
3. Distributed Computing: Fundamentals, Simulations, and Advanced Topics, by Hagit Attiya and Jennifer Welch.
4. Randomized Algorithms, by Rajeev Motwani and Prabhakar Raghavan.
5. Principles of Distributed Computing, lecture notes by Roger Wattenhofer.

High Speed Networks (ECSE222)

Credit: 3+0

Course Objective:

- Students will get an introduction about ATM and Frame relay.
- Students will be provided with an up-to-date survey of developments in High-Speed Networks.
- Enable the students to know techniques involved to support real-time traffic and congestion control.
- Students will be provided with different levels of quality of service (QoS) to different applications.

Syllabus

- 1. High Speed Networks** Frame relay networks – Asynchronous transfer mode – ATM protocol architecture – ATM logical connection– ATM cell – ATM service categories – AAL – High speed LANs – Fast ethernet, gigabit ethernet – Fiber channel – Wireless LANs– Applications – Requirements – Architecture of 802.11.
- 2. Congestion and Traffic Management** Queuing analysis – Queuing models – Single server queues – Effects of congestion – Congestion control – Traffic management – Congestion control in packet switching networks – Frame relay congestion control.
- 3. TCP and ATM Congestion Control** TCP flow control – TCP congestion control – Retransmission – Timer management – Exponential RTO backoff – KARN's algorithm – Window management – Performance of TCP over ATM – Traffic and congestion control in ATM – Requirements – Attributes, Traffic management frame work – traffic control – ABR traffic management – ABR rate control – RM cell formats – ABR capacity allocation – GFR traffic management.
- 4. Integrated and Differentiated Services & Protocols For QOS Support RSVP** Integrated services architecture – Approach, components, services – Queuing discipline, FQ – PS – BRFQ – GPS – WFQ – Random early detection, differentiated services. **Protocols for QOS SUPPORT RSVP** Goals and characteristics – data flow – RSVP operations – Protocol mechanisms – Multi protocol label switching – Operations – Label stacking – Protocol details – RTP – Protocol architecture – Data transfer protocol – RTCP.

Course Outcome (CO): Upon completion of the course, the students will be able to

CO1: Compare and analyze the fundamental principles of various high speed communication networks and their protocol architectures.

CO2: Analyze the methods adopted for performance modeling of traffic flow and estimation.

CO3: Examine the congestion control issues and traffic management in TCP/IP and ATM networks.

CO4: Compare, analyze and implement the various routing protocols in simulation software tools.

CO5: Examine the various services.

Text Books:

1. Jean warland and Pravin Wadaja, "HIGH PERFORMANCE COMMUNICATION NETWORKS", 2nd Edition, Jean Harcourt Asia Pvt. Ltd.,2001.
2. William Stallings, "High Speed Networks and Internet", 2nd Edition, Pearson Education, 2002.
3. Irvan Pepelnjk, Jim Guichard and Jeff Apcar, "Mpls and Vpn Architecture", Volume 1 and 2, Cisco Press, 2003

Software Defined Network (ECSE223)

Credit: 3+0

Course Objective: This course focuses on fundamentals of Software Defined Networks and its application.

Syllabus

- 1. Introduction:** History of Software Defined Networking (SDN) – Modern Data Center – Traditional Switch Architecture – Why SDN – Evolution of SDN – How SDN Works – Centralized and Distributed Control and Data Planes
- 2. Open Flow & SDN Controllers:** Open Flow Specification – Drawbacks of Open SDN, SDN via APIs, SDN via Hypervisor- Based Overlays – SDN via Opening up the Device – SDN Controllers – General Concepts. **Data Centers** Multitenant and Virtualized Multitenant Data Center – SDN Solutions for the Data Center Network – VLANs – EVPN – VxLAN – NVGRE
- 3. SDN Programming Programming SDNs:** Northbound Application Programming Interface, Current Languages and Tools, Composition of SDNs – Network Functions Virtualization (NFV) and Software Defined Networks: Concepts, Implementation and Applications
- 4. SDN :**Juniper SDN Framework – IETF SDN Framework – Open Daylight Controller – Floodlight Controller – Bandwidth Calendaring – Data Center Orchestration

Course Outcome (CO): Upon completion of the course, the students will be able to

CO1: Analyze the evolution of software defined networks

CO2: Express the various components of SDN and their usage.

CO3: Explain the use of SDN in the current networking scenario.

CO4: Design and develop various applications of SDN.

Text Books

1. Paul Goransson and Chuck Black, —Software Defined Networks: A Comprehensive Approach, First Edition, Morgan Kaufmann, 2014.
2. Thomas D. Nadeau, Ken Gray, —SDN: Software Defined Networks, O'Reilly Med

Pervasive Computing (ECSE224)

Credit: 3+0

Course Objective: This course provides a fundamental theoretical concept in Pervasive Computing and its applications in various areas.

Syllabus

1. **Pervasive Computing Concepts:** Perspectives of Pervasive Computing, Challenges, Technology; The Structure and Elements of Pervasive Computing Systems: Infrastructure and Devices, Middleware for Pervasive Computing Systems, Pervasive Computing Environments
2. **Context Collection, User Tracking, and Context Reasoning:** Resource Management in Pervasive Computing: Efficient Resource Allocation in Pervasive Environments, Transparent Task Migration, Implementation and Illustrations.
3. **HCI Interface in Pervasive Environments:** HCI Service and Interaction Migration, Context-Driven HCI Service Selection, Scenario Study: Video Calls at a Smart Office, A Web Service– Based HCI Migration Framework.
4. **Pervasive Mobile Transactions:** Mobile Transaction Framework, Context-Aware Pervasive Transaction Model, Dynamic Transaction Management, Formal Transaction Verification, Evaluations

Course Outcome (CO): At the end of course, the student will be able to

CO1: Understand the fundamental theoretical concepts in pervasive computing.

CO2: Understand the aspects of context awareness.

CO3: Study the methods for efficient resource allocation and task migration.

CO4: Learn and Analyze the HCI Service Selection and HCI migration framework.

CO5: Design and implement pervasive application systems.

Text Books:

1. Minyi Guo, Jingyu Zhou, Feilong Tang, Yao Shen," Pervasive Computing: Concepts, Technologies and Applications",CRC Press, 2016.
2. Obaidat, Mohammad S., Mieso Denko, and Isaac Woungang, eds. Pervasive computing and networking. John Wiley & Sons, 2011.
3. Laurence T. Yang, Handbook on Mobile and Ubiquitous Computing Status and Perspective, 2012, CRC Press.

Middleware Technologies (ECSE225)

Credit: 3+0

Course Objective: The objective of the course is to teach the role of middleware in the distributed environment and its common services.

Syllabus

1. **Client / Server Concepts** Client – Server – File Server, Database server, Group server, Object server, Web server. Middleware – General middleware – Service specific middleware. Client / Server Building blocks – RPC – Messaging – Peer – to- Peer.
2. **EJB Architecture EJB** EJB Architecture – Overview of EJB software architecture – View of EJB – Conversation – Building and Deploying EJBs – Roles in EJB.
3. **EJB Applications** EJB Session Beans – EJB entity beans – EJB clients – EJB Deployment – Building an application with EJB.
4. **CORBA & COM:** CORBA – Distributed Systems – Purpose – Exploring CORBA alternatives – Architecture overview – CORBA and networking model – CORBA object model – IDL – ORB – Building an application with CORBA. **COM** COM – Data types – Interfaces – Proxy and Stub – Marshalling – Implementing Server / Client – Interface Pointers – Object Creation, Invocation, Destruction – Comparison COM and CORBA – Introduction to .NET – Overview of .NET architecture – Marshalling – Remoting.

Course Outcome (CO): At the end of course, the student will be able to

CO1: To study the set of services that a middleware system constitutes of.

CO2: To understand how middleware facilitates the development of distributed applications in heterogeneous environments.

CO3: To study how it helps to incorporate application portability, distributed application component interoperability and integration.

CO4: To learn the object-oriented middleware basics through the example of the following CORBA objects.

CO5: To understand the basics of Web services that is the most oft-used middleware technique.

Text Books:

1. Robert Orfali, Dan Harkey and Jeri Edwards, "The Essential Client/Server Survival Guide", Galgotia Publications Pvt. Ltd., 2002.
2. Tom Valesky, "Enterprise Java Beans", Pearson Education, 2002.
3. Jason Pritchard, "COM and CORBA side by side", Addison Wesley, 2000.
4. Jesse Liberty, "Programming C#", 2nd Edition, O'Reilly Press, 2002.
5. Mowbray, "Inside CORBA", Pearson Education, 2002.
6. Jeremy Rosenberger, "Teach yourself CORBA in 14 days", Tec media, 2000.

Guidelines for Project Work

Project (CSEP312)

Credit: 0+3

Problem Statement for the project should be carefully written to meet out the COs given below.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Develop a technical artifact requiring new technical skills and effectively utilizing a new software tool to complete a task.

CO2: Write requirements documentation, selecting appropriate technologies, identifying and creating appropriate test cases for systems.

CO3: Demonstrate understanding of professional customs & practices and working with professional standards.

CO4: Improve problem-solving, critical thinking skills and report writing.

CO5: Learn professional skills like exercising leadership, behaving professionally, behaving ethically, listening effectively, and participating as a member of a team, developing appropriate workplace attitudes.

Note: CSEP312 may be extended as CSEP402.

Project (CSEP402)

Credit: 0+3

Problem Statement for the project should be carefully written to meet out the COs given below.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Analyze and understand the real-life problem and apply their knowledge to get programming solution.

CO2: Engage in the creative design process through the integration and application of diverse technical knowledge and expertise to meet customer needs and address social issues.

CO3: Use the various tools and techniques, coding practices for developing real life solution to the problem.

CO4: Find out the errors in software solutions and establishing the process to design maintainable software applications.

CO5: Write the report about what they are doing in project and learning the team working skills.

Note: CSEP402 must be extended as CSEP403.

Project (CSEP403)

Credit: 0+6

Problem Statement for the project should be carefully written to meet out the COs given below.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Analyze and understand the real-life problem and apply their knowledge to get programming solution.

CO2: Engage in the creative design process through the integration and application of diverse technical knowledge and expertise to meet customer needs and address social issues.

CO3: Use the various tools and techniques, coding practices for developing real life solution to the problem.

CO4: Find out the errors in software solutions and establishing the process to design maintainable software applications.

CO5: Write the report about what they are doing in project and learning the team working skills.

Note: CSEP403 must be concluded meaningfully.

**Syllabus
of
Open Elective Subjects Offered by the Department**

Web Technology (CSEOE01)

Course Objective: The course content enables students to learn and design web application.

Credit: 3+0

Syllabus

1. **Introduction:** Introduction and Web Development Strategies, History of Web and Internet, Protocols Governing Web, Writing Web Projects, Connecting to Internet, Introduction to Internet services and tools, Introduction to client-server computing. Core Java: Introduction, Operator, Data type, Variable, Arrays, Methods & Classes, Inheritance, Package and Interface, Exception Handling, Multithread programming, I/O, Java Applet, String handling, Event handling, Introduction to AWT, AWT controls, Layout managers.
2. **Web Page Designing, Scripting and HTML:** List, Table, Images, Frames, forms, CSS, Document type definition, XML: DTD, XML schemes, Object Models, presenting and using XML, Using XML Processors: DOM and SAX, Dynamic HTML. Scripting: Java script: Introduction, documents, forms, statements, functions, objects; introduction to AJAX, Networking: Internet Addressing, InetAddress, Factory Methods, Instance Methods, TCP/IP Client Sockets, URL, URL Connection, TCP/IP Server Sockets, Datagram.
3. **Servlets:** Servlet Overview and Architecture, Interface Servlet and the Servlet Life Cycle, Handling HTTP get Requests, Handling HTTP post Requests, Redirecting Requests to Other Resources, Session Tracking, Cookies, Session Tracking with Http Session Java Server Pages (JSP): Introduction, Java Server Pages Overview, A First Java Server Page Example, Implicit Objects, Scripting, Standard Actions, Directives, Custom Tag Libraries.JDBC.
4. **JSP:** Introduction to JSP, Implicit Objects, JSP Scripting Tag, JSP Descriptive Tag, JSP Custom Tag, JDBC.

Course Outcome (CO): At the end of course, the student will be able to

CO1: Explain web development Strategies and Protocols governing Web.

CO2: Develop Java programs for window/web-based applications.

CO3: Design web pages using HTML, XML, CSS and JavaScript.

CO4: Creation of client-server environment using socket programming.

CO5: Building enterprise level applications and manipulate web databases using JDBC.

CO6: Design interactive web applications using Servlets and JSP.

Text Books:

1. Burdman, Jessica, "Collaborative Web Development" Addison Wesley
2. Xavier, C, "Web Technology and Design", New Age International
3. Ivan Bayross, "HTML, DHTML, Java Script, Perl & CGI", BPB Publication
4. Bhave, "Programming with Java", Pearson Education
5. Herbert Schildt, "The Complete Reference: Java", McGraw Hill.
6. Hans Bergsten, "Java Server Pages", SPD O'Reilly

7. Margaret Levine Young, "The Complete Reference Internet", McGraw Hill.
8. Naughton, Schildt, "The Complete Reference JAVA2", McGraw Hill.
9. Balagurusamy E, "Programming in JAVA", McGraw Hill.

Web Application Development using Python (CSEOE02)

Credit: 3+0

Course Objective:

- To learn the core programming features of Python
- To understand the back-end data model and accessing
- To understand Django fundamentals and use its concepts to build and deploy robust web applications and apps.
- Learn about Django URL patterns and views and deploy Django applications.

Syllabus

1. **Python Basics:** Standard types -Flow Control – Exception handling – Files -Functions- Strings-List- Tuples- Dictionaries & Object-oriented programming.
2. **DJANGO Creating** the Project – Running the Development Server – Creating the Application – Designing a Model – Setting up the Database – Setting up the Application – Dynamic Web Sites – Communication – Data Storage – Presentation.
3. **DJANGO Architecture Django and Python** - Django stake on MVC: Models - Views and Template – Overall Django Architecture - 3 Core Files: models.py- urls.py- views.py – URL's – Modeling HTTP: Requests; Responses and Middleware – Views / Logic.
4. **DJANGO Forms and APIs** Templates – Forms - Validation – Authentication - Advanced Forms processing techniques – Django REST framework – Django piston.

Course Outcome (CO): After completing these course- students will have a fundamental understanding of how-to Implementing Python based applications

CO1: Safely model and store data in SQL databases

CO2: Use MVC (Models- Views & Templates)

CO3: Build and deploy robust Django web apps

CO4: Integrate with RESTful web services

CO5: Unit Testing and Debugging Django apps

Text Books:

1. Python Web Development with Django 1st Edition by Jeff Forcier; Paul Bissex; Wesley Chun; Pearson Education; 2009.
2. Think Python, 2nd Edition - How to Think Like a Computer Scientist - Allen B. Downey – 2nd Edition, Shroff / O'Reilly Publication, 2016.
3. Django for Beginners: Build websites with Python and Django Paperback, William S Vincent, Independently Published, 2018.

Front End Technologies (CSEOE03)

Credit: 3+0

Course Objective:

- To provide an introduction to the principles and practices of Network Engineering.
- To understand the architecture of the network devices.
- To learn QoS related methodologies.
- To explore the emerging technologies in network engineering.

Syllabus

1. **HTML5 & CSS3** – Introduction, Elements, Tags, Lists, Tables, Images, Forms - Form Elements & Attributes, Hidden Fields, Semantic Elements, Media Elements, Canvas, SVG, Drag & Drop, Geolocation, Web Storage, Special Tags, Formatting Tags. CSS – Introduction, Styling, Box Model, Padding & Dimension, Transforms, Transitions, Animations, Multiple columns, User Interface
2. **Bootstrap** - Overview, Environment setup, Precompiled Bootstrap, Source Code, Grid System, Bootstrap CSS Overview, Typography, Code, Tables, Forms, Helper Classes, Responsive Utilities, Glyphicons, Dropdowns, Navigation Elements, Breadcrumb, Pagination, Badges, Progress bars, Plugins Overview, Transition Plugin, Modal Plugin, Dropdown Plugin, Scrollspy Plugin, Tab Plugin, Tooltip Plugin, Popover Plugin, Alert Plugin, Button Plugin, Collapse Plugin, Carousel Plugin, Affix Plugin.
3. **Java Script** - Introduction, Variable, Value, Data type, Operators and Expressions, Arrays, Decision making and Loops, Functions, Variable Scope and Objects, The Browser Environment – BOM & DOM, Events and Event Handling.
4. **JQUERY**- Introduction, Structure, Using jQuery and including .js file to HTML, Type of Selectors, Handling Events with jQuery, jQuery DOM Traversal, jQuery DOM Manipulation, jQuery Effects, APIs, Geo-location, Local Storage, Session Storage, History.

Course Outcome (CO): Upon completion of the course, the students should be able to

CO1: understand and describe the role of front-end development in modern web applications.

CO2: act like a professional front-end developer.

CO3: Able to analyze problems, seek for needed information, apply a solution, and verify it.

CO4: See the alternative ways for creating a front-end.

CO5: Able to discuss and take into use more front-end technologies.

Text Books:

1. Web Design with HTML, CSS, JavaScript and jQuery Set Paperback by Jon Duckett, 2014.
2. Mastering HTML, CSS & Javascript Web Publishing Paperback – 15 Jul 2016 by Laura Lemay, Rafe Colburn, Jennifer Kyrnin
3. Beginning HTML, XHTML, CSS, and JavaScript (Wrox Programmer to Programmer) Paperback – Import, 29 Dec 2009 by Jon Duckett
4. HTML 5 Black Book (Covers CSS3, JavaScript, XML, XHTML, AJAX, PHP, jQuery)

Back-End Technologies (CSEOE04)

Credit: 3+0

Course Objective:

- To learn the basics of Angular JS
- To implement Forms, inputs and Services
- To implement Directives and Databases
- To understand basics of Nodejs and Express
- To understand the basics of Mongo DB and its data model

Syllabus

- 1. Introduction to Angular JS:** Introduction – Features – Angular JS Model-View-Controller – Expression -Directives and Controllers - Angular JS Modules – Arrays –Working with ng-model – Working with Forms – Form Validation – Error Handling with Forms – Nested Forms with ng-form – Other Form Controls.
- 2. Directives & Building Databases:** Filters – Using Filters in Controllers and Services – Angular JS Services – Internal Angular JS Services – Custom Angular JS Services - Directives – Alternatives to Custom Directives – Understanding the Basic options – Interacting with Server –HTTP Services – Building Database, Front End and Back End.
- 3. Node JS and Express Framework:** Introduction –Using the Terminals – Editors –Building a Webserver with Node – The HTTP Module – Views and Layouts –Middle ware – Routing – Form Handling with Express - The Request and Response Objects –Handle bars – Comments and Blocks.
- 4. Introduction to MONGODB:** JSON and MongoDB – Adopting a Non-relational Approach – Opting for Performance vs. Features Running the Database Anywhere – Generating or Creating a Key – Using Keys and Values – Implementing Collections **DATA MODELS:** Understanding Databases – Reviewing the Feature List - Designing the Database – Building Indexes – Inserting Data – Querying for Data – Updating Data – Removing Data – Referencing a Database – GRID FS.

Course Outcome (CO): Upon completion of the course, the students should be able to

CO1: Implement Directives and Controllers.

CO2: Create Forms, validate and use Filters.

CO3: Understand the usage of Keys and Values.

CO4: Implementations of data model.

Text Books:

1. Adam Freeman - ProAngular JS, Apress, First Edition, 2014.
2. ShyamSeshadri, Brad Green –“AngularJS: Up and Running: Enhanced Productivity with structured Web Apps”, Apress, O'Reilly Media, Inc.
3. AgusKurniawan–“AngularJS Programming by Example”, First Edition, PE Press, 2014.
4. David Hows, Peter Membrey, EelcoPlugge – “MongoDB Basics”, Apress, 2014.
5. Ethan Brown, “Web Development with Node and Express”, Oreilly Publishers, First Edition.